



Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Hálózati Rendszerek és Szolgáltatások Tanszék

Posztkvantum-kriptográfia alkalmazása az IoT rendszerek biztonságának megerősítésére

SZAKDOLGOZAT

Készítette
Király Róbert

Konzulens
Gódor Győző

SZAKDOLGOZAT FELADAT

Király Róbert

Mérnök-informatikus hallgató részére

Posztkvantum-kriptográfia alkalmazása az IoT rendszerek biztonságának megerősítésére

A posztkvantum-kriptográfia a kvantumszámításhoz kapcsolódó kutatások egyik kulcsterülete, amely olyan kriptográfiai algoritmusok fejlesztésére irányul, amelyek ellenállnak a kvantumszámítógépekkel kivitelezhető támadásoknak. E megoldások célja, hogy hagyományos, mai hardvereken is hatékonyan futtathatóak legyenek.

Az IoT (Internet of Things) eszközök mára az internetes infrastruktúra meghatározó elemei, amelyek gyakran erőforrás-korlátozott környezetben működnek (korlátozott memória, alacsony számítási kapacitás). Mivel ezek az eszközök sok esetben érzékeny adatokat kezelnek, különösen fontos a biztonságos, ugyanakkor hatékony kriptográfiai megoldások alkalmazása.

A hallgató feladat a National Institute of Standards and Technology (NIST) által ajánlott posztkvantum algoritmusok vizsgálata IoT környezetben. Az elemzés során értékelni kell az algoritmusok teljesítményét CPU-, memória- és energiafelhasználás szempontjából, valamint biztosítani kell a megfelelő biztonsági szintet. A cél egy olyan optimalizált kompromisszum meghatározása, amely egyensúlyt teremt a hatékonyság és a kriptográfiai biztonság között.

A hallgató feladatának a következőkre kell kiterjednie:

- Jelölt mutassa be a posztkvantum-kriptográfia alapelveit és jelentőségét, különös tekintettel a kvantumszámítógépek által jelentett kriptográfiai fenyegetésekre!
- Röviden mutassa be IoT rendszerek architektúráis sajátosságait és erőforrás-korlátozásait!
- Ismertesse a NIST által ajánlott posztkvantum-kriptográfiai algoritmusokat, azok matematikai hátterét és biztonsági tulajdonságait!
- Jelölt implementálja a kiválasztott algoritmusokat virtuális IoT környezetbe, megfelelő fejlesztői eszközök és platformok alkalmazásával!
- Végezzen méréseket az algoritmusok teljesítményének értékelésére, különös tekintettel a futási időre, memóriahasználatra és energiafogyasztásra!
- Az eredmények alapján értékelje az algoritmusok IoT környezetben való alkalmazhatóságát!

Tanszéki konzulens: Gódor Győző

Külső konzulens:

Budapest, 2026. március 19.

Dr. Bacsárdi László
egyetemi docens
tanszékvezető

Tartalomjegyzék

Kivonat	i
Abstract	ii
1. Bevezetés	1
1.1. A kriptográfia és az információbiztonság rövid története	1
1.2. Kvantumszámítógépek	3
1.2.1. A kvantumszámítás működési elvei	3
1.2.2. A kvantumszámítógépek előnyei és lehetőségei	4
1.2.3. Veszélyek a jelenlegi kriptográfiai rendszerekre	4
1.3. A dolgok internete	5
1.3.1. Az IoT kialakulása és fejlődése	5
1.3.2. IoT eszközök jellemzői és korlátai	6
1.3.3. IoT biztonsági kihívások	6
1.4. Posztkvantum-kriptográfia és IoT – a kapcsolat	7
1.5. A dolgozat céljai és felépítése	8
2. Posztkvantum-kriptográfiai protokollok	9
2.1. Posztkvantum-kriptográfia matematikai alapjai	9
2.1.1. A rácselmélet alapjai	9
2.1.2. Az LWE probléma és annak moduláris változata	10
2.2. CRYSTALS-Kyber protokoll	10
2.2.1. Paraméterek és biztonsági szintek	10
2.2.2. Kulcscsere és kulcsencapsulation	11
2.2.2.1. Kulcsgenerálás (KeyGen)	11
2.2.2.2. Üzenet becsomagolása (Encaps)	11
2.2.2.3. Üzenet kicsomagolása (Decaps)	12
2.2.3. Biztonsági tulajdonságok és hardver gyorsulás	12
2.3. BIKE (Bit Flipping Key Encapsulation) protokoll	12
2.3.1. Kód-alapú titkosítás alapelvei	13
2.3.2. MDPC kódok és az BIKE paraméterek	13
2.3.3. BIKE kulcsgenerálás és encapsulation	13
2.3.3.1. Kulcsgenerálás (KeyGen)	13
2.3.3.2. Üzenet becsomagolása (Encaps)	14
2.3.3.3. Üzenet kicsomagolása (Decaps) és bit-flipping dekódolás	14
2.3.4. Biztonsági tulajdonságok	15
2.4. Oldalsávós támadások és biztonsági implementáció	15
2.4.1. Időzítési támadások (Timing Attacks)	15
2.4.2. Energiafogyasztási támadások (Power Analysis)	15
2.4.3. Hibabeviteli támadások (Fault Injection)	16
2.5. IoT alkalmazások és integráció	16

2.5.1.	Kyber IoT implementáció	16
2.5.2.	BIKE IoT implementáció	16
2.5.3.	Hibrid megközelítés az IoT-ben	16
2.5.4.	Memória- és tárolási követelmények	17
2.6.	Kyber és BIKE összehasonlítása	17
2.6.1.	Biztonsági megfontolások	17
2.6.2.	Teljesítmény- és méretjellemzők	17
2.6.3.	Gyakorlati alkalmazások	17
2.6.4.	Hibrid megközelítés	18
3.	Mérési környezet	19
3.1.	Hardverkörnyezet	19
3.1.1.	Tesztplatform specifikációja	19
3.1.2.	Hálózati topológia	19
3.1.3.	Az adatfolyam és a titkosítás helye	20
3.1.4.	Energiaellátás és mérőeszközök	21
3.2.	Szoftverkörnyezet	21
3.2.1.	Operációs rendszer és alapkönyvtárak	21
3.2.2.	Kriptográfiai könyvtárak	21
3.2.3.	MQTT infrastruktúra	22
3.2.4.	Konténerizáció és reprodukálhatóság	22
3.2.5.	A Prometheus és a Grafana szerepe a mérési infrastruktúrában	23
3.3.	A mérési környezet összefoglalása	23
4.	Mérési metodológia	25
4.1.	A kísérlet tervezési elvei	25
4.1.1.	Szimulált erőforrás-korlátok	25
4.1.2.	Kísérletmátrix és ismétlések	25
4.2.	A mért paraméterek	26
4.2.1.	TLS kézfogás késleltetése	26
4.2.2.	MQTT publish-ciklus késleltetése	26
4.2.3.	Üzenetátviteli sebesség	26
4.2.4.	PUBACK-visszaigazolás és időtúllépés-nyomon követés	26
4.2.5.	Folyamatszintű memóriefoglalás	27
4.2.6.	Processzorkihasználtság	27
4.3.	Adatgyűjtési és futtatási eljárás	27
4.3.1.	Automatizált kísérletfuttatás	27
4.3.2.	Metrikák offline exportálása	28
4.3.3.	Statisztikai módszer	28
4.4.	A mérési megközelítés korlátai	28
5.	Mérések	29
5.1.	MQTT publish-ciklus késleltetése – X25519 (klasszikus referencia)	29
5.2.	ML-KEM algoritmusok MQTT mérési eredményei	29
5.2.1.	ML-KEM-512	30
5.2.2.	ML-KEM-768	31
5.2.3.	ML-KEM-1024	31
5.3.	X25519+ML-KEM-768 (hibrid)	32
5.4.	BIKE algoritmusok mérési eredményei	32
5.4.1.	BIKE-L1	32
5.4.2.	BIKE-L3	34

5.4.3. BIKE-L5	34
5.5. Memóriahasználat	35
5.6. Az átlagos p_{99} késleltetés összefoglalása	35
6. Eredmények kiértékelése	37
6.1. Teljesítmény összehasonlítása	37
6.1.1. MQTT publish-ciklus késleltetése: ML-KEM-768 és X25519	37
6.1.2. BIKE-L1 operatív instabilitása	39
6.1.3. Memóriaigény	40
6.2. Biztonsági szempontú értékelés	40
6.2.1. Biztonsági szintek és NIST kategóriák	40
6.2.2. Ismert kriptanalitikai eredmények	40
6.3. Alkalmazhatósági elemzés	41
6.3.1. Erőforrás-korlátozott eszközökre vonatkozó következtetések	41
6.3.2. Protokoll-szintű kihívások	41
6.3.3. ML-KEM-768 és BIKE-L1 összehasonlító értékelése	42
6.4. Az eredmények összefoglalása	43
7. Összefoglalás és javaslatok	44
7.1. A dolgozat eredményeinek összefoglalása	44
7.2. Javaslatok IoT rendszerek PQC migrációjához	45
7.2.1. Algoritmusválasztás: ML-KEM-768 az ajánlott alapértelmezés	45
7.2.2. Hibrid KEM az átmeneti időszakra	45
7.2.3. Cryptographic agility a firmware-tervezésben	45
7.2.4. MQTT protokoll-szintű megfontolások	45
7.3. Jövőbeli kutatási irányok	46
7.3.1. Valódi ARM-hardveren végzett mérések	46
7.3.2. Energiafogyasztás mérése	46
7.3.3. PQC-aláírás az IoT tanúsítványláncban	46
7.3.4. Hardveres NTT-gyorsítás és alvó módú optimalizálás	46
Irodalomjegyzék	47
Függelék	48
F.1. Nyilatkozat generatív mesterséges intelligencia alkalmazásáról	48
F.2. A laborkörnyezet forrásfájljai	50

HALLGATÓI NYILATKOZAT

Alulírott *Király Róbert*, szigorló hallgató kijelentem, hogy ezt a szakdolgozatot meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem. A Függelékben egyértelműen megjelöltem, hogy a mesterséges intelligencia eszközeit alkalmaztam-e a dolgozat elkészítéséhez; amennyiben igen, annak módját és mértékét a táblázatban közöltem. Tudomásul veszem, hogy a mesterséges intelligenciával generált tartalomért – annak mértékétől függetlenül – teljes felelősséggel tartozom.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövege pedig a BME Címtárban regisztrált személyek számára elérhető legyen. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Budapest, 2026. június 7.

Király Róbert
hallgató

Kivonat

A kvantumszámítógépek fejlődése közvetlen fenyegetést jelent a jelenleg széles körben alkalmazott nyilvános kulcsú kriptográfiai rendszerekre. Az IoT-eszközök különösen sérülékenyek, mivel hosszú üzemidejük alatt a ma rögzített titkosított forgalom egy jövőbeli kvantumszámítógéppel visszafejthető lehet – ezt a *harvest now, decrypt later* támadási modell írja le. Dolgozatomban azt vizsgálom, hogy a NIST által 2024-ben szabványosított ML-KEM (CRYSTALS-Kyber, FIPS 203) posztkvantum kulcscsere-algoritmus alkalmazható-e erőforrás-korlátozott IoT-eszközök MQTT alapú kommunikációjának védelmére anélkül, hogy az elfogadható késleltetési határokat átlépné.

A kísérleti platform Docker Compose alapú, konténerizált labor, amelyben három szimulált IoT-csomópont stunnel sidecar proxyn, PQC-TLS 1.3 kapcsolaton keresztül publikál MQTT QoS 1 üzeneteket egy Eclipse Mosquitto brokernek. A CPU-korlátot öt szinten állítottam be (0,026 –1,0 mag), amelyek sorban az ESP32, a Raspberry Pi Zero 2W, a 3B+, a 4B és egy ipari átjáró számítási teljesítményét modellezzik. Az összes mért metrikát – köztük a publish-ciklus késleltetési hisztogramokat, az újracsatlakozásszámot és az RSS-memóriát – Prometheus gyűjtötte, Grafana vizualizálta.

A mérések négy fő eredményt hoztak. Egyrészt az ML-KEM-768 és az X25519 steady-state MQTT publish-késleltetése szignifikánsan nem különbözik. A p_{50} mindkét esetben $\approx 1,5$ ms, a p_{99} 1,0 magon 7–8 ms. Másrészt a CPU-szint erős küszöbhatást mutat: 0,25 mag felett a p_{95} 2–5 ms alá esik, míg 0,026 magon 166–235 ms-ra nő – ez a stunnel proxy CPU-éhezéséhez, nem a PQC-algoritmus számítási igényéhez köthető. Valamint a BIKE (QC-MDPC alapú) algoritmuscsalád minden vizsgált szinten (L1, L3, L5) teljes PUBACK-hiányt és rendszeres kapcsolatfelbomlást mutatott, ami a jelenlegi stunnel architektúrával nem kompatibilis. Végül azt figyeltem meg, hogy az IoT-csomópont memóriahasználata (RSS ≈ 32 MB) algoritmusfüggetlen, mivel a PQC-számítás a proxy oldalán zajlik.

Az eredmények alapján az ML-KEM-768 jól implementált, telepíthető megoldás az IoT MQTT kommunikáció kvantumbiztos védelmére, feltéve, hogy a biztonsági proxy legalább Raspberry Pi 3B+ szintű CPU-kapacitással rendelkezik.

Abstract

The advancement of quantum computing poses a direct threat to widely deployed public-key cryptographic systems. IoT devices are particularly vulnerable: due to their long operational lifetimes, encrypted traffic captured today may be decrypted by a future quantum computer – a threat model known as *harvest now, decrypt later*. This thesis investigates whether ML-KEM (CRYSTALS-Kyber, NIST FIPS 203), the post-quantum key encapsulation mechanism standardised in 2024, can protect MQTT-based communication of resource-constrained IoT devices without exceeding acceptable latency bounds.

The experimental platform is a Docker Compose laboratory in which three simulated IoT nodes publish MQTT QoS 1 messages to an Eclipse Mosquitto broker over PQC-TLS 1.3 connections, mediated by a stunnel sidecar proxy. CPU limits were set at five levels (0.026–1.0 cores), modelling the compute capacity of an ESP32, Raspberry Pi Zero 2W, 3B+, 4B, and an industrial gateway, respectively. All metrics – including publish-cycle latency histograms, reconnect counts, and RSS memory – were collected by Prometheus and visualised in Grafana.

Four main findings emerged. First, the steady-state MQTT publish latency of ML-KEM-768 and X25519 does not differ significantly. The p_{50} is ≈ 1.5 ms for both, and the p_{99} at 1.0 core is 7–8 ms. Second, CPU level exhibits a pronounced threshold effect: above 0.25 cores the p_{95} falls below 2–5 ms, while at 0.026 cores it rises to 166–235 ms – attributable to CPU starvation of the stunnel proxy, not to the PQC algorithm itself. Furthermore, the BIKE (QC-MDPC-based) algorithm family exhibited complete PUB-ACK loss and frequent reconnections at all tested security levels (L1, L3, L5), rendering it incompatible with the current stunnel architecture. Finally, I observed that IoT node memory usage (RSS ≈ 32 MB) is algorithm-independent, as all PQC computation is offloaded to the proxy.

The results demonstrate that ML-KEM-768 is a well-implemented, deployable solution for quantum-safe protection of IoT MQTT communication, provided the security proxy has at least Raspberry Pi 3B+-level CPU capacity.

1. fejezet

Bevezetés

Az emberiség kommunikációjának védelme iránti igény egyidős a civilizációval. Az írott információ titkosítása évezredek óta folyamatosan fejlődő diszciplína, amelyet minden kor szak saját technológiai szintjéhez igazított. A 20. század végén lezajlott digitális forradalom azonban minőségileg új helyzetet teremtett: a kriptográfia többé nem pusztán katonai vagy diplomáciai eszköz, hanem a globális digitális infrastruktúra alapköve. Bankrendszerek, egészségügyi nyilvántartások, ipari vezérlőrendszerek és hétköznapi kommunikáció egyaránt a nyilvános kulcsú kriptográfia védelmére támaszkodik. E rendszerek biztonsága matematikai problémák számítási nehézségére épül – egy feltevésre, amelynek érvényessége a közeljövőben komolyan megkérdőjeleződhet.

Párhuzamosan a digitális forradalom kibontakozásával egy másik technológiai változás is lezajlott: a dolgok internete (Internet of Things, IoT) robbanásszerű terjedése. Beágyazott processzorok, érzékelők és hálózati kapcsolat ma már nem csupán számítógépekben, hanem ipari gépekben, orvosi implantátumokban, okos otthonokban és kritikus infrastruktúrában is megtalálható. A becslések szerint 2030-ra több mint 25 milliárd hálózatra kötött eszköz lesz forgalomban [1], amelyek mindegyike érzékeny adatokat generál, tárol és továbbít, miközben erőforrásaikban – számítási teljesítményben, memóriában és energiában – erősen korlátozottak.

A két tendencia találkozásánál egy harmadik, egyre fenyegetőbb fejlemény jelenik meg: a kvantumszámítógépek fejlődése. A kvantummechanika elvein alapuló számítási paradigma alkalmas arra, hogy a ma használt nyilvános kulcsú kriptográfiai rendszerek matematikai alapjait polinomiális időn belül megoldja. Ez azt jelenti, hogy egy kellően erős kvantumszámítógép megjelenésével a jelenlegi titkosítási infrastruktúra teljes egészében sérülékennyé válik – nem csupán a jövőbeli kommunikáció, hanem a ma titkosított és elmentett forgalom is, amelyet a jövőben visszafejtenek.

A dolgozat célja, hogy megvizsgálja, hogy hogyan lehet posztkvantum-kriptográfiai (PQC) megoldásokat – konkrétan a CRYSTALS-Kyber és a BIKE kulcskapszulázó algoritmusokat – hatékonyan alkalmazni erőforrás-korlátozott IoT környezetekben. A bevezető fejezet az elméleti háttérrel tárgyalja: bemutatja a kriptográfia fejlődésének történetét, részletezi a kvantumszámítógépek működési elvét, előnyeit és veszélyeit, majd ismerteti az IoT ökoszisztéma sajátosságait és biztonsági kihívásait, végül bemutatja a PQC és IoT területek kapcsolatát.

1.1. A kriptográfia és az információbiztonság rövid története

A titkosítás legkorábbi ismert példái az ókori Egyiptomból származnak, ahol Kr. e. 1900 körül módosított hieroglifákat alkalmaztak a szöveg értelmének elrejtésére. Az ókori Görögországban a szkütalé nevű hengerrel transzpozíciós titkosítást végeztek, míg Julius Caesar

a nevét viselő Caesar-rejtjellel titkosította katonai üzeneteit: minden betűt az ábécé harmadik következő betűjével helyettesített. Ezek az ún. klasszikus rejtjelek helyettesítésen vagy transzpozíción alapultak, azonban a kriptanalízis fejlődésével rendszeresen feltörhetővé váltak.

A középkorban a polialfabetikus rejtjelezés megjelenése (Vigenère-rejtjel, 1553) egy lépéssel előrébb vitte a titkosítás nehézségét, ám a Kasiski-teszt (1863) és a Friedman-koincidenziindex (1920-as évek) módszereivel ez a megközelítés is feltörhetőnek bizonyult. A klasszikus kriptográfia lényegi korlátja az volt, hogy minden rendszer biztonsága a kulcs titkosságán alapult, miközben a kulcsot valahogyan biztonságosan el kellett juttatni a kommunikáló felekhez – ez az ún. kulcsterjesztési probléma, amely évszázadokig megoldhatatlannak tűnt.

A modern kriptográfia alapjait a 20. század első felében fektették le. A második világháborúban az Enigma-gép és az annak feltörésére irányuló bletchley-i erőfeszítések döntő szerepet játszottak a konfliktus kimenetelében, és egyben megalapozták a számítástechnika és az algoritmikus kriptográfia fejlődését. Alan Turing, Gordon Welchman és munkatársaik az elektromechanikus Bombe gépek segítségével automatizálták az Enigma-kulcsok feltörését, és ezzel a kriptanalízist ipari méretűvé tették. Claude Shannon 1949-es munkája, a *Communication Theory of Secrecy Systems* matematikai alapokra helyezte az információelmélet és a kriptográfia kapcsolatát: bevezette az entrópia, a tökéletes titkosság és a biztonság formális fogalmait. Shannon megmutatta, hogy tökéletes titkosság egyetlen rendszertől, az egyszer használatos rejtjeltől (one-time pad) várható el, amely azonban kulcskezelési okokból nem praktikusan alkalmazható.

Az 1970-es évek hoztak igazi forradalmat. 1973-ban az USA Nemzeti Szabványügyi Intézete (NBS, ma NIST) meghirdette az egységes adattitkosítási szabvány pályázatát, amelyből 1977-ben a DES (Data Encryption Standard) lett. A DES egy 64 bites blokkméretű, 56 bites kulcsú szimmetrikus rejtjel, amely az IBM Lucifer algoritmusán alapult, és amelynek tervezésébe az NSA is beleszólt. A DES évtizedekig ipari szabvány maradt, ám 1998-ban az EFF (Electronic Frontier Foundation) *Deep Crack* nevű dedikált hardverével 56 óra alatt visszafejtett egy DES-titkosított üzenetet, egyértelműen bizonyítva az algoritmus elavultságát. Utódja, a 2001-ben szabványosított AES (Advanced Encryption Standard) – Vincent Rijmen és Joan Daemen Rijndael algoritmusán alapulva – máig az egyik legelterjedtebb szimmetrikus titkosítási algoritmus.

Az igazi paradigmaváltást azonban a nyilvános kulcsú kriptográfia 1976-os bevezetése jelentette. Whitfield Diffie és Martin Hellman *New Directions in Cryptography* című munkájukban [2] egy merőben új elvet fogalmaztak meg: a titkos kulcs megosztása nélkül is lehetséges biztonságos kommunikáció létrehozása. Ez az ún. aszimmetrikus kriptográfia alapelve: egy nyilvános kulccsal bárki titkosíthat, de visszafejteni csak a megfelelő privát kulcs birtokosa képes. A Diffie–Hellman kulcscsere a diszkrét logaritmus problémán alapul, amelynek megoldása a csoport megfelelő megválasztásával praktikusan kivitelezhetetlennek tűnt.

Ezt az ötletet rövidesen Ron Rivest, Adi Shamir és Leonard Adleman ültette át a gyakorlatba az RSA algoritmus formájában (1977, közzétéve 1978-ban), amelynek biztonsága a nagy számok faktorizálásának nehézségén alapul. Az 1980-as és 1990-es évtizedben Neal Koblitz és Victor Miller egymástól függetlenül javasolta az elliptikus görbe kriptográfia (ECC) alkalmazását, ahol a csoportként elliptikus görbék pontjait alkalmazzák. Az ECC kisebb kulcsmérettel nyújt azonos biztonsági szintet az RSA-hoz képest, ami mobil és beágyazott alkalmazásokban komoly előnnyé vált.

A 2000-es évektől a kriptográfia az internet mindennapi infrastruktúrájává vált. Az új protokolloknak már alapkövetelménye a megfelelő szintű titkosítás. A titkosítást nem alkalmazó megoldásokat folyamatosan vezetik ki az internet forgalmából. Mára elképzelhetetlennek tűnik egy olyan weboldal, amely nem használ HTTPS-t, vagy egy e-mail szol-

gátlató, amely nem támogatja a TLS-t. A VPN-ek, a biztonságos fájl tárolás és a digitális aláírások mind a nyilvános kulcsú kriptográfia eredményei, amelyek nélkülözhetetlenek a modern digitális élethez. Azonban ez a globális infrastruktúra most szembesül azzal a fenyegetéssel, hogy a kvantumszámítógépek megjelenésével gyakorlatilag azonnal elavulttá válik.

1.2. Kvantumszámítógépek

A kvantumszámítógépek nem egyszerűen gyorsabb klasszikus számítógépek: egy alapvetően más számítási paradigmát képviselnek, amely a kvantummechanika sajátos jelenségeit – a szuperpozíciót, az összefonódást és az interferenciát – használja fel számítási erőforrásként. Ennek megértése nélkülözhetetlen a kriptográfiai következmények értékeléséhez.

1.2.1. A kvantumszámítás működési elvei

A klasszikus számítástechnika alapegysége a bit, amely kizárólag a 0 vagy az 1 értéket veheti fel. A kvantumszámítógép alapegysége a kvantumbit, röviden qubit, amelynek állapotát a kvantummechanika szabályai írják le. A qubit állapotát egy kétdimenziós komplex Hilbert-tér elemeként ábrázolhatjuk legkönnyebben a Bloch-gömb segítségével. Egy qubit általános állapota egy szuperpozíció, amely a következőképpen írható fel:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

ahol $\alpha, \beta \in \mathbb{C}$ és $|\alpha|^2 + |\beta|^2 = 1$. Ez azt jelenti, hogy mérés előtt a qubit egyszerre tartalmazza a $|0\rangle$ és a $|1\rangle$ állapotot, az ún. szuperpozíció elvének megfelelően. Mérés hatására a szuperpozíció összeomlik: $|\alpha|^2$ valószínűséggel kapjuk a 0-t, $|\beta|^2$ valószínűséggel az 1-et.

A szuperpozíció önmagában még nem tenné hatékonyvá a kvantumszámítást. A másik kulcsfontosságú jelenség az összefonódás (entanglement): két qubit összefonódott állapotban olyan módon korrelál, hogy az egyik mérési eredménye azonnali hatással van a másikra, függetlenül a köztük lévő fizikai távolságtól. Az összefonódás következménye, hogy n qubit nem $2n$, hanem 2^n állapot egyidejű reprezentációjára képes. Egy 300 qubit rendszer több lehetséges állapotot reprezentál egyszerre, mint ahány atom van az ismert univerzumban.

A harmadik alapjelenség az interferencia. A kvantumalgoritmusokat úgy tervezik, hogy a helyes választ adó amplitúdók konstruktívan interferáljanak (erősítsék egymást), a helytelen válaszokat adó amplitúdók pedig destruktívan (gyengítsék egymást). Ez a mechanizmus teszi lehetővé, hogy a kvantumalgoritmusok exponenciálisan nagy keresési tereken belül hatékonyan találják meg a megoldást. A kvantumalgoritmus lényegében az amplitúdókat manipulálja kvantumkapukkal (unitér transzformációkkal), majd a végállapot mérésével nyeri ki a hasznos információt.

A kvantumszámítógépek fizikai megvalósítása számos különböző technológiai úton halad. A szupravezető qubitek (amelyeket az IBM, Google és mások fejlesztenek) mesterséges atomokat alkalmaznak szupravezető Josephson-kontaktuson alapuló áramkörökben, kriogén hőmérsékleten (≈ 15 mK). A csapdázott ionok (IonQ, Quantinuum) elektromágneses csapdáknak tartott ionokat használnak qubitként, ami hosszabb koherenciaidőt és alacsonyabb hibatartalmat eredményez. A fotonikus és topologikus kvantumszámítógépek szintén kutatás alatt állnak. 2019-ben a Google bejelentette a kvantumfölény elérését: a Sycamore processzor 53 qubittal egy mintavételi feladatot 200 másodperc alatt oldott meg, amelyhez a legjobb klasszikus szuperszámítógépek becsléseik szerint 10 000 évet igényeltek volna [3]. 2023-ra az IBM Eagle és Heron processzorai már 127, illetve 133 qubitet tartalmaztak, és a hibajavítással ellátott logikai qubitek is megjelentek.

A kvantumhiba-javítás a skálázhatóság egyik legnagyobb kihívása. Fizikai qubitek hibával terheltek: a koherenciaidő véges, a kapuhiba valószínűsége 10^{-3} – 10^{-4} nagyságrendű. A hibajavított logikai qubitek eléréséhez becslések szerint 1 000–10 000 fizikai qubit szükséges egyetlen logikai qubitenként, ami azt jelenti, hogy egy kriptográfiailag releváns, néhány ezer logikai qubitet igénylő számítás elvégzéséhez millió fizikai qubit méretű gép szükséges. Ez ma még nem áll rendelkezésre, de a fejlesztési ütem gyors.

1.2.2. A kvantumszámítógépek előnyei és lehetőségei

A kvantumszámítástechnika potenciális alkalmazásai messze túlmutatnak a kriptográfiai fenyegetésen, és számos területen ígéri a klasszikus számítástechnika fundamentális korlátainak áttörését.

A **molekulaszimulációk és kvantumkémia** területén a kvantumszámítógépek képesek molekulák elektronszerkezetét kvantummechanikai szinten szimulálni. Míg klasszikus számítógépen egy közepes méretű molekula pontos szimulációja exponenciálisan növekvő erőforrást igényel, addig egy kvantumszámítógép természetes hatókörébe esik a feladat. Ez közvetlen alkalmazást ígér a gyógyszerfejlesztésben (hatóanyag–fehérje kölcsönhatások), az anyagtudományban (új szupravezetők, akkumulátor anyagok tervezése) és a katalízisvizsgálatban.

Optimalizálási feladatok esetén a kvantum heuristikák – mint a Quantum Approximate Optimization Algorithm (QAOA) és a Quantum Annealing – NP-nehéz problémák közelítő megoldására mutatnak ígéretes eredményeket. Logisztika, ellátásilánc-tervezés, pénzügyi portfólió-optimalizálás és menetrendtervezés területén már kisebb méretű kísérletekben is demonstráltak kvantum előnyt.

A **pénzügyi modellezésben** a Monte Carlo-szimulációk kvantum gyorsítása négyzetgyök-csökkentést ígér a mintaszámban, ami kockázatbecslés és derivatív árazás területén jelent versenyelőnyt. Az **mesterséges intelligencia** területén kvantum-neuronhálózatokat és kvantum kernel-módszereket vizsgálnak, bár itt az előny kevésbé egyértelmű, mint az előző területeken.

Végül meg kell említeni a **kvantumkulcs-elosztást (QKD)**, amely a kvantummechanika törvényein alapuló biztonságos kulcscserét kínál: a kvantumállapotok lemásolhatatlansága (no-cloning tétel) és a mérés rendelkező hatása garantálja, hogy a lehallgatás kimutatható. A QKD azonban saját infrastruktúrát és speciális hardvert igényel, és nem közvetlenül helyettesíti a PQC-t.

1.2.3. Veszélyek a jelenlegi kriptográfiai rendszerekre

A kvantumszámítógépek kriptográfiai fenyegetése két alapvető algoritmuson keresztül válik nyilvánvalóvá.

A Shor-algoritmus: 1994-ben Peter Shor bizonyította, hogy egy kellően erős kvantumszámítógép polinomiális idő alatt képes az egészsfaktorizáció és a diszkrét logaritmus problémájának megoldására [4]. Az algoritmus kvantum-Fourier-transzformációt alkalmaz az adott szám prímtényezőinek meghatározásához. A faktorizálás időbonyolultsága $O((\log N)^3)$, szemben a legjobb ismert klasszikus szub-exponenciális algoritmussal, az általános számmező szitával (GNFS), amely körülbelül $O(\exp((\log N)^{1/3}))$ nagyságrendű. Ez közvetlenül aláássa az RSA, a Diffie–Hellman kulcscsere és az ECC biztonságát: az RSA-2048 feltöréséhez hozzávetőleg 4 000 hibajavított logikai qubit szükséges – ez a szám a jelenlegi fejlesztési ütem alapján akár a 2030-as évekre el is érhető.

A Grover-algoritmus: 1996-ban Lov Grover megmutatta, hogy egy N elemű rendezetlen adatbázisban a keresés kvantumszámítógépen $O(\sqrt{N})$ lépésben elvégezhető, szemben a klasszikus $O(N)$ lépéssel. Szimmetrikus kriptográfiánál ez a négyzetgyök-gyorsulás

azt jelenti, hogy egy 128 bites AES kulcs bruteforce-keresése olyan, mintha 64 bites kulcsra kellene keresni: a megoldás a kulcsméret megduplázása, azaz az AES-256 alkalmazása. A szimmetrikus kriptográfia tehát módosítással kvantum-ellenálló maradhat.

A legjelentősebb stratégiai veszélyt azonban a „**gyűjtsd most, fejtsd meg később**” (harvest-now, decrypt-later, HNDL) stratégia jelenti. Állami szintű szereplők és fejlett kiberbűnözői csoportok már most gyűjthetik a titkosított hálózati forgalmat, hogy azt egy jövőbeli kvantumszámítógéppel visszafejtsék. Azok az adatok, amelyeknek ma titkosítva kell maradniuk 10–20 éves időtávon (állami titkok, egészségügyi adatok, szellemi tulajdon, pénzügyi tranzakciók), már most célponttá válnak. Ez a fenyegetés sürgetővé teszi a PQC bevezetését még azelőtt, hogy a kvantumszámítógépek kriptográfiai szintre érnek.

A NIST 2016-ban meghirdetett versenye ennek a felismerésnek a közvetlen következménye volt. A hat éves értékelési folyamat eredményeként 2022-ben a NIST közzétette az első négy posztkvantum-kriptográfiai algoritmust: a CRYSTALS-Kyber-t (kulcs-kapszulázás), a CRYSTALS-Dilithium-ot és a FALCON-t (digitális aláírás), valamint a SPHINCS+-t (hash-alapú aláírás) [5]. 2024-ben ezek FIPS szabványként is megjelentek (FIPS 203, 204, 205).

1.3. A dolgok internete

Az IoT fogalma egy mélyen heterogén ökoszisztémát takar, amelynek határai az évek során folyamatosan tágulnak. Az egyszerű otthoni okoseszközöktől az ipari automatizálás vezérlőrendszerein át az orvosi implantátumokig terjed az eszközök köre, amelyek összekötik a fizikai és a digitális világot.

1.3.1. Az IoT kialakulása és fejlődése

Az „összes dolgot hálózatba kötni” ötlete valójában megelőzi az „Internet of Things” elnevezést. Mark Weiser, a Xerox PARC kutatója 1991-es *The Computer for the 21st Century* esszéjében lefektette az ubiquitous computing (mindenütt jelenlévő számítástechnika) koncepcióját: elképzelte, hogy számítástechnikai eszközök a mindennapi tárgyakba integrálódnak, és a felhasználó számára láthatatlanul végzik munkájukat. Az 1990-es évek elején a Carnegie Mellon Egyetemen már létezett az első hálózatba kötött automatával bemutatott kísérlet, amely szenzorok segítségével tájékoztatta a felhasználókat az üdítőgép aktuális tartalmáról.

A „dolgok internete” elnevezést Kevin Ashton brit technológus alkotta meg 1999-ben, eredetileg az RFID (rádiófrekvenciás azonosítás) és az ellátásilánc-menedzsment összefüggésében. Az elgondolás lényege, hogy ha minden fizikai tárgyhoz egyedi azonosítót és hálózati kapcsolatot rendelünk, az objektumok maguk is adatforrásokká válnak, és a valós világ állapota automatikusan nyomon követhetővé válik. Az RFID technológia lehetővé tette például a raktári készletezés automatizálását, amelyet a Procter & Gamble és az Auto-ID Center együttműködésével fejlesztett ki.

A 2000-es évek elejétől az ipari szenzorhálózatok (Wireless Sensor Networks, WSN) terjedtek el a gyártásban, a mezőgazdaságban és a katonai megfigyelésben. A fordulópontot az okostelefonok megjelenése (2007), az IPv6 szabvány széleskörű bevezetése, valamint az alacsony fogyasztású rádióprotokollok (Bluetooth LE, ZigBee, LoRaWAN, NB-IoT) és az olcsó ARM-alapú mikrovezérlők tömeges piaci megjelenése hozta el. 2010-től az IoT eszközök száma exponenciálisan növekedett: 2015-re elérte a 15 milliárdot, 2020-ra meghaladta a 30 milliárdot.

Az intelligens otthonok piacán a Nest termosztát (2011), az Amazon Echo (2014) és a Philips Hue izzósorozat paradigmaváltást hozott a fogyasztói elektronikában. Az ipari IoT a gyártásban és az energetikában a prediktív karbantartás, a folyamatoptimalizálás és a

valós idejű monitorozás eszközévé vált. Az egészségügyi IoT területén a viselhető szenzoros eszközöktől a beültetett pacemaker-eken és szívritmus-figyelőkön át az infúziós pumpák hálózati vezérléséig terjedt az alkalmazás. A McKinsey Global Institute becslése szerint az IoT gazdasági hatása 2030-ra 5,5–12,6 billió dollárt tesz ki évente [6].

1.3.2. IoT eszközök jellemzői és korlátai

Az IoT ökoszisztéma egyik legfontosabb sajátossága a rendkívüli heterogenitás és az erőforrás-korlátok jelenléte. Egy nagyvállalati szerver és egy ipari hőmérséklet-érzékelő között több nagyságrendnyi különbség lehet számítási teljesítményben, memóriában és energiaellátásban.

Tipikus korlátok az alacsony kategóriás IoT eszközöknél:

- **Processzor:** 8–32 bites mikrovezérlők (pl. ARM Cortex-M0/M4, Atmel AVR), órajel 8–120 MHz tartományban, hardveres lebegőpontos egység nélkül.
- **Memória:** 2–256 kB RAM, néhány kB – néhány MB flash tároló.
- **Energia:** elemről működő eszközöknél az aktív kriptográfiai számítás az egyik legnagyobb energiaigényű művelet, ezért fontos a kulcscsere-műveletek ritkítása.
- **Sávszélesség:** sok protokoll (LoRaWAN, NB-IoT) esetén a payload mérete néhány tíz-száz bájt-ra korlátozott, ami a PQC kulcs- és kapszulaméreteit érzékenyen érinti.
- **Üzemidő:** ipari és infrastrukturális eszközöket 10–20 éves élettartamra terveznek, ami azt jelenti, hogy a ma telepített eszközök a kvantumfenyegetés kritikus időszakában is aktívan üzemelni fognak.

Ez utóbbi szempont különösen fontos. Egy 2024-ben telepített, 15 éves élettartamra tervezett ipari szenzor 2039-ig működik. Ha a 2030-as évekre valóban megjelennek kriptográfiailag releváns kvantumszámítógépek, ez az eszköz az aktív üzemeltetési ideje alatt válik sebezhető célponttá. Az ún. *cryptographic agility* – a kriptográfiai algoritmusok futás közbeni cserélhetőségének megőrzése a rendszerarchitektúrában – ezért az IoT tervezés egyik alapkövetelményévé vált.

1.3.3. IoT biztonsági kihívások

Az IoT biztonsági kérdései már jóval a kvantumfenyegetés előtt komoly figyelmet kaptak. A 2016-os Mirai botnet-támadás megmutatta, hogy hétköznapi IoT eszközök (IP-kamerák, DVR-ek) gyári jelszóval hagyott eszközei milyen pusztító erejű DDoS-támadáshoz nyújthatnak alapot: a botnet csúcsán másodpercenként 620 Gbps forgalmat generált, és az internetes infrastruktúra jelentős részét megbénította [7]. A 2021-es Oldsmar vízkezelő-üzemi incidens, ahol egy támadó a klórszint módosításával próbálkozott, jól illusztrálta az IoT rendszerek fizikai következményekkel járó sebezhetőségét.

A klasszikus kriptográfiai fenyegetéseken túl az IoT specifikus biztonsági kihívásai a következők:

Hitelesítési problémák. Erőforrás-korlátozott eszközökön a PKI alapú tanúsítványkezelés (TLS 1.3 futtatása, tanúsítványmegújítás, visszavonás-ellenőrzés) számottevő CPU-terhelést és memóriát igényel. Sok eszköz ezért gyári, nehezen cserélhető, megosztott titkokra támaszkodik, ami egy pontos sebezhetőséget teremt.

Fizikai hozzáférés és oldalsáv-támadások. Az IoT eszközök fizikailag hozzáférhető helyen üzemelnek, fizikai biztonságuk korlátozott. A fogyasztásmérés-alapú (Differential Power Analysis, DPA) és elektromágneses sugárzáson alapuló (Electromagnetic Analysis,

EMA) elemzések titkok kinyerésére alkalmasak, hacsak az implementáció nem állandó idejű (constant-time) és nem alkalmaz megfelelő zaj-ellenintézkedéseket.

Firmware-frissítések és cryptographic agility. A hosszú üzemidő miatt az algoritmusváltásnak szoftverfrissítésen keresztül kell megvalósulnia. Ez előre megtervezett biztonságos bootloadereket, aláírt firmware-csomagokat és megbízható frissítési csatornákat feltételez. Ezek hiányában a deployed eszközök nem frissíthetők kvantum-ellenálló algoritmusokra.

Kulcskezelés és üzenetméret. Az IoT rendszerek skálája – sok milliós eszközpark – egyedi kulcsmenedzsment kihívásokat vet fel. A PQC implementáció során figyelemmel kell lenni arra, hogy a rács alapú és kód alapú algoritmusok nyilvános kulcsai és kapszulái lényegesen nagyobbak, mint az RSA vagy ECC megfelelőik, ami ütközhet az eszközök korlátozott tárhelyével és a sávszélességkorlátokkal.

1.4. Posztkvantum-kriptográfia és IoT – a kapcsolat

A posztkvantum-kriptográfia azokat a kriptográfiai algoritmusokat foglalja magában, amelyek ellenállnak mind a klasszikus, mind a kvantumszámítógépek által nyújtott támadásoknak. Elnevezése némileg félrevezető: a „poszt-” előtag nem azt jelenti, hogy a kvantumszámítógépek már megjelentek, hanem azt, hogy az algoritmusok a kvantumos utáni korszakra is biztonságosak.

A PQC főbb matematikai megközelítései:

- **Rács alapú kriptográfia:** az SVP és az LWE probléma nehézségén alapul. Ide tartoznak a CRYSTALS-Kyber (KEM) és CRYSTALS-Dilithium (aláírás), amelyeket a NIST 2022-ben szabványosított [5].
- **Kód alapú kriptográfia:** lineáris hibakijavító kódok dekódolásának NP-nehézségén alapul. A BIKE és a HQC ide sorolható; a McEliece-rendszer 1978 óta ismert, de kulcsméretei igen nagyok.
- **Hash alapú kriptográfia:** kiforrottsága és konzervatív biztonsági feltevései miatt kisebb aláírási rendszerek esetén vonzó (SPHINCS+).
- **Izogénian alapuló kriptográfia:** a SIKE algoritmust 2022-ben klasszikus támadással feltörték, ami e kategória korlátjaira hívja fel a figyelmet.

Az IoT kontextusában a PQC bevezetése különleges kihívásokat jelent. A CRYSTALS-Kyber-512 esetén a nyilvános kulcs 800 bájt, a kapszula 768 bájt, míg az RSA-2048 esetén ez 256 bájt – a háromszoros méretbeli növekedés mind a sávszélességet, mind a flash-tárhelyt igénybe veszi. Az elterjedt TLS és DTLS protokollok üzenetméreteinek módosítására is szükség van.

A számítási igény tekintetében kedvezőbb a kép: a CRYSTALS-Kyber és a BIKE ARM Cortex-M4 processzorral néhány tíz-száz milliszekundum alatt elvégezhetők [8, 9], ami a legtöbb IoT alkalmazásban elfogadható. A **hibrid megközelítés** – klasszikus és PQC kulcs csere párhuzamos alkalmazása – a biztonsági migráció kockázatát is csökkenti: a kommunikáció biztonságos marad mindaddig, amíg legalább az egyik algoritmus ellenáll a fenyegetésnek. Ez ma az iparági ajánlás a PQC fokozatos bevezetésekor.

Az ipari és szabályozói nyomás is egyre erősebb. Az Európai Unió NIS2-irányelve és a Cyber Resilience Act kötelezővé teszik a beágyazott rendszerek gyártói számára a biztonsági frissíthetőség biztosítását és a kvantum-ellenálló kriptográfiára való tervezett átállást. Az IETF aktívan dolgozik a PQC-t integráló TLS és DTLS profilok kidolgozásán, különösen a CoAP és a 6LoWPAN kombinálásával működő IoT-profilok esetén [10].

Az összefonódó kihívások – a kvantumfenyegetés, az IoT korlátok és a szabályozói nyomás – együttesen sürgetik a PQC algoritmusok erőforrás-tudatos implementációinak fejlesztését. E dolgozat éppen erre a hiányra ad választ: a CRYSTALS-Kyber és a BIKE algoritmusok IoT kontextusban való teljesítményének részletes mérésével és elemzésével.

1.5. A dolgozat céljai és felépítése

A dolgozat fő kutatási kérdése: *A CRYSTALS-Kyber és BIKE posztkvantum-kriptográfiai algoritmusok erőforrás-korlátozott IoT eszközökön való alkalmazhatóságának és egymáshoz viszonyított teljesítménybeli különbségeinek meghatározása szimulált mérési körülmények között.*

A kutatási célkitűzés megvalósításához a következő részfeladatokat tűztem ki:

1. A CRYSTALS-Kyber és a BIKE algoritmusok elméleti hátterének és matematikai alapjainak részletes ismertetése, különös tekintettel az IoT-szempontról releváns implementációs kihívásokra.
2. Mérési környezet kialakítása, amely lehetővé teszi a két algoritmus szimulált erőforrás-korlátozott eszközön való teljesítmény-összehasonlítását.
3. A kulcsgenerálás, a kulcskapszulázás (encapsulation) és a visszafejtés (decapsulation) futási idejének, memóriaigényének és energiaszükségletének mérése és kiértékelése.
4. Következtetések levonása arról, melyik algoritmus alkalmasabb egyes IoT alkalmazási területekre, és milyen paraméterbeállítások optimálisak a biztonság és az erőforrásigény közötti kompromisszum szempontjából.

2. fejezet

Posztkvantum-kriptográfiai protokollok

Jelen fejezetben részletesen bemutatom a posztkvantum-kriptográfia két legígéretesebb protokollját, a CRYSTALS-Kyber és a BIKE (Bit Flipping Key Encapsulation) algoritmusokat. Ezek a protokollok a modern kriptográfiai kihívásokra adnak választ, előkészítve a rendszereinket a kvantumszámítógépek jövőbeli fenyegetésére. A fejezet célja, hogy megértsük a matematikai alapokat, a működési mechanizmusokat és az alkalmazási lehetőségeket, különös tekintettel az integrált rendszerek (IoT) biztonságára.

2.1. Posztkvantum-kriptográfia matematikai alapjai

A posztkvantum-kriptográfia azokra a kriptográfiai algoritmusokra utal, amelyek ellenállnak a kvantumszámítógépek támadásainak. A klasszikus kriptográfia biztonsága olyan matematikai problémák nehézségén alapul, amelyek klasszikus számítógépeken véges idő alatt nem oldhatók meg. Azonban a Shor-algoritmus felfedezése óta ismert, hogy a kvantumszámítógépek exponenciálisan felgyorsíthatják a faktorizációs és diszkrét logaritmus problémák megoldását, ami az RSA és az elliptikus görbe kriptográfia alapjait veszélyezteti.

A posztkvantum-kriptográfiai algoritmusok olyan matematikai struktúrára épülnek, amelyeknél még nem sikerült kvantum gyorsulást találni. Az egyik legfontosabb ilyen struktúra a rácselméleten alapuló titkosítás, amely a modul Learning With Errors (module-LWE) problémára épül.

2.1.1. A rácselmélet alapjai

A rács egy olyan diszkrét algebrai struktúra, amely a lineáris algebra és a számelmélet metszéspontjában helyezkedik el. Formálisan, egy n dimenziós rács a vektortérnek egy olyan részhalmaza, amely egy n vektorból álló bázis összes egész lineáris kombinációjaként állítható elő. Legyen $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n$ egy rács bázisa, akkor a rács pontjaira teljesül:

$$\Lambda = \left\{ \sum_{i=1}^n c_i \mathbf{b}_i \mid c_i \in \mathbb{Z} \right\}$$

A rácsok alapvető jellemzője, hogy számos nehéz matematikai probléma társul hozzájuk. Az egyik leghíresebb a legrövidebb vektor probléma (Shortest Vector Problem, SVP), amely azt a választ keresi, hogy egy adott rácson, mekkora a nullvektortól különböző legrövidebb vektor hossza. A másik központi probléma a legközelebbi vektor probléma (Closest Vector Problem, CVP), amely egy adott vektor rácshoz legközelebbi rácspontját

nak megkeresésére vonatkozik. Mindkét probléma NP-nehéznek bizonyult, és jelenleg nem ismeretes polinomiális idejű megoldása még kvantumszámítógépeken sem.

Az LWE probléma ezt az ötletet természetes módon a kriptográfiára adaptálja. Egy támadó abban az esetben sikeres, ha egy lineáris rendszer megoldását tudja helyreállítani úgy, hogy az egyenletek mellett hibás mérések is rendelkezésre állnak. Ez a probléma általánosítható moduláris polinomokra, amit module-LWE-nek neveznek, és ez képezi az alapját a Kyber protokollnak.

2.1.2. Az LWE probléma és annak moduláris változata

Az LWE (Learning With Errors) problémát Regev 2005-ben vezette be. Az alapfeladat a következőképpen fogalmazható meg: Tegyük fel, hogy van egy titok vektor $\mathbf{s} \in \mathbb{Z}_q^n$, és meg szeretnénk szerezni információt róla. Erre példa az $\langle \mathbf{a}, \mathbf{s} \rangle + e \pmod{q}$ egyenletek egy sorozata, ahol $\mathbf{a}$ egy véletlen vektor $\mathbb{Z}_q^n$ -ből, és e egy kis hiba $\mathbb{Z}_q$ -ből egy χ eloszlásból (tipikusan egy Gauss-eloszlás a tangens függvény kombinációjában).

Az LWE probléma azt keresi, hogy polinomiális időben eldönthető-e, hogy egy $(\mathbf{A}, \mathbf{b})$ pár, ahol $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$, vagy pedig a $\mathbf{b}$ csak egy véletlen vektor. Ez a döntési verzió, de létezik keresési verziója is, ahol az $\mathbf{s}$ vektort kell helyreállítani.

A Kyber protokoll a module-LWE problémát használja, amely az LWE-nek egy hatékonyabb változata. Ebben az esetben a vektorok helyett polinomokkal dolgozunk egy gyűrűben. A gyűrű általában $R_q = \mathbb{Z}_q[X]/(X^n + 1)$, ahol n kettő hatványa és q egy nagyobb prímszám. Az $X^n + 1$ polinomot választjuk, mert a NTT (Number Theoretic Transform) algoritmussal hatékonyan lehet vele számolni.

A module-LWE probléma a következőképpen írható fel: adott az $\mathbf{A} \in R_q^{k \times \ell}$ egy véletlen mátrix polinomokkal, az $\mathbf{s} \in R_q^\ell$ egy titokvektor és az $\mathbf{e} \in R_q^k$ egy kis hibavektor, a feladat az $(\mathbf{A}, \mathbf{b})$ pár megkülönböztetése egy $(\mathbf{A}, \mathbf{b}')$ párotól, ahol $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$, $\mathbf{b}'$ pedig egyenletesen véletlen. Az LWE és module-LWE problémákat széles körben tanulmányozták, és jelenleg semmilyen klasszikus vagy kvantumalgoritmus nem ismert, amely polinomiális időben megoldaná őket.

2.2. CRYSTALS-Kyber protokoll

A CRYSTALS-Kyber (Crystal Lattice Signatures for Encryption and Signatures - Kyber) a posztkvantum-kriptográfia egyik legígéretesebb megoldása. A protokoll 2022-ben standardizálva lett a NIST által, és széles körben használható a gyakorlati alkalmazásokban. A Kyber egy kulcstokozási mechanizmus (Key Encapsulation Mechanism, KEM), amely lehetővé teszi, hogy két fél közösen megosztott titkot hozzon létre, amely később szimmetrikus titkosításhoz használható.

2.2.1. Paraméterek és biztonsági szintek

A Kyber protokoll különböző biztonsági szinteket kínál, amely az alkalmazás igényeihez igazítható. Az alap paraméterek:

- **Kyber512:** 128 bites klasszikus biztonsági szint, amely az AES-128 szintű biztonságnak felel meg
- **Kyber768:** 192 bites klasszikus biztonsági szint, amely az AES-192 szintű biztonságnak felel meg
- **Kyber1024:** 256 bites klasszikus biztonsági szint, amely az AES-256 szintű biztonságnak felel meg

Az egyes szintekhez különböző paraméterek tartoznak. A Kyber512 a következő paramétereket használja (ahol k az $\mathbf{A}$ mátrix dimenziója):

- A Kyber512 paraméterek: $n = 256$, $q = 3329$, $k = 2$, $\eta_1 = 2$, $\eta_2 = 2$.
- A Kyber768 paraméterek: $n = 256$, $q = 3329$, $k = 3$, $\eta_1 = 2$, $\eta_2 = 2$.
- A Kyber1024 paraméterek: $n = 256$, $q = 3329$, $k = 4$, $\eta_1 = 2$, $\eta_2 = 2$.

Az η_1 és η_2 paraméterek határozzák meg a hibavektorok egy középére igazított binomiális eloszlásának szórásának szintjét. Ezek az értékek kritikusak a biztonsági szint garantálásához, mert a túl kicsi hibák a protokollt törhetővé teszik, míg a túl nagy hibák viszont a decapsulation megbízhatóságát csökkentik.

2.2.2. Kulcscsere és kulscencapsulation

A Kyber protokoll három fő szubrutinból áll: **KeyGen**, **Encaps**, és **Decaps**.

2.2.2.1. Kulcsgenerálás (KeyGen)

A kulcsgenerálás során az egyik fél (Alice) elkészíti a nyilvános és privát kulcsot. Az algoritmus a következő lépésekből áll:

1. Sorsoljunk le egy véletlenszerű értéket $d \in \{0, 1\}^{32}$ (ezt a seed-et)
2. A d seed-ből válassz ki az ρ értéket a SHA3-512 hash függvényvel
3. Az ρ értékből generáljunk egy véletlen mátrixot $\mathbf{A} \in R_q^{k \times k}$ az XOF (extendable output function) beépített generátorral
4. Generáljunk egy véletlenszerű secretet $\mathbf{s}, \mathbf{e} \in R_q^k$ binomiális eloszlásból az η_1 paraméterrel
5. Számítsuk ki a $\mathbf{t} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \in R_q^k$ értéket
6. A nyilvános kulcs: $pk = (\mathbf{t}, \rho)$
7. A privát kulcs: $sk = (\mathbf{s}, \mathbf{e})$, valamint további technikai adatok

Az algoritmus során az $\mathbf{A}$ mátrix számítása a kiterjesztett kimeneti függvény segítségével történik, amely biztosítja, hogy az mátrix pszeudó-véletlen marad, de reprodukálható.

2.2.2.2. Üzenet becsomagolása (Encaps)

A Kyber encapsulation során egy kulcscsatorna készül. Bob, aki Alice nyilvános kulcsát ismeri, a következő lépéseket hajtja végre:

1. Sorsoljunk le egy véletlenszerű üzenet $m \in \{0, 1\}^{32}$ -t
2. A m üzenetből válassz ki G hash függvényvel egy $(r, d') \in \{0, 1\}^{32} \times \{0, 1\}^{32}$ párt
3. Generáljunk egy véletlenszerű vektort $\mathbf{y} \in R_q^k$ binomiális eloszlásból az η_1 paraméterrel
4. Számítsuk ki a $\mathbf{u} = \mathbf{A}^T \cdot \mathbf{y} + \mathbf{e}' \in R_q^k$ értéket, ahol $\mathbf{e}' \in R_q^k$ binomiális eloszlásból az η_2 paraméterrel

5. Számítsuk ki a $\mu = \lfloor \frac{q}{2} \rfloor \cdot m_i$ értéket minden i -re, és a $v = \mathbf{t}^T \cdot \mathbf{y} + e'' + \mu$ értéket, ahol e'' egy skalár hibavektor
6. A kapszula (ciphertext): $c = (\mathbf{u}, v)$
7. A megosztott titok (shared secret): $K = H(m)$, ahol H a SHA3-256 függvény

Az encapsulation folyamata garantálja, hogy az üzenet m titkos marad, mivel csak Alice, aki ismeri az $\mathbf{s}$ vektort, tudja helyreállítani az eredeti üzenetet.

2.2.2.3. Üzenet kicsomagolása (Decaps)

Az Alice által végrehajtott decapsulation az encapsulation inverze. A nyilvános kapszula és a privát kulcs alapján:

1. Számítsuk ki a $\mathbf{u}'$ és v' értékeket az $\mathbf{s}$ vektort használva: $\mathbf{u}' = \mathbf{s}^T \cdot \mathbf{u} \in R_q^k \pmod{q}$
2. Számítsuk ki a hibahatáron belüli üzenet biteket: $m'_i = 1$ ha $|v - \mathbf{u}' - \lfloor \frac{q}{2} \rfloor|_q > \lfloor \frac{q}{4} \rfloor$, egyébként $m'_i = 0$
3. Az helyreállított üzenet: $m' = \text{recover}(\mathbf{u}', v)$
4. A megosztott titok: $K = H(m')$

Az LWE probléma nehézsége biztosítja, hogy az $\mathbf{s}$ vektor nélkül nem lehet helyreállítani az üzenetet, még akkor sem, ha az encapsulation alatt zajló számítások részleges információi rendelkezésre állnak.

2.2.3. Biztonsági tulajdonságok és hardver gyorsulás

A Kyber biztonsága a module-LWE probléma feltételezett nehézségén alapul. A NIST szabványosítási folyamatban az algoritmusokkal szembeni támadások részletes elemzésére irányuló kutatások azt mutatják, hogy a Kyber protokoll az elvárt biztonsági szinteket nyújtja, ha a paraméterek helyesen kerülnek megválasztásra. A Kyber különösen sérülékeny az oldalsávós támadásokra (side-channel attacks), így az implementációnak óvatosságnak kell lennie az időzítési információk elrejtésekor.

A Kyber nagy előnye az NTT algoritmusnak köszönhető, amely a gyűrűn végzett szorzásokat lineáris időre csökkenti. Az NTT a Fast Fourier Transform (FFT) módosított verziója, amely egész számok fölött működik a q prímszám feldolgozásával. A modern processzorok (mint az Intel vagy ARM) támogatják a vektoros utasításokat (SIMD - Single Instruction Multiple Data), amelyekkel a Kyber hatékonyan felgyorsítható. Az optimalizált implementációk másodpercenként több száz kulcskeresést tudnak végezni standard laptop processzorokon.

2.3. BIKE (Bit Flipping Key Encapsulation) protokoll

A BIKE egy alternatív posztkvantum-kriptográfiai protokoll, amely az error-correcting codes elméletén alapul. Szemben a Kyber rács-alapú megközelítésével, a BIKE kód-alapú kriptográfiát alkalmaz, amely a McEliece rendszer eszméire épül. A kód-alapú titkosítás hosszú történelmi gyökerekkel rendelkezik, és erős ellenálló képességgel bír a kvantum-támadásokkal szemben.

2.3.1. Kód-alapú titkosítás alapelvei

A kód-alapú kriptográfia fundamentális ötlete, hogy az error-correcting kódok dekódolása általános esetben NP-nehéz probléma. Az egyik legfontosabb eredmény, hogy szinte minden véletlen kód dekódolása NP-teljes. Ez azt jelenti, hogy nem ismeretes polinomiális idejű algoritmus az általános eset dekódolására, még kvantum-támadások alatt sem.

A McEliece rendszer, amely 1978-ban lett bevezetve, ezt az ötletet használta. A titkos kulcs az egy könnyen dekódolható kód (például Goppa-kód), a nyilvános kulcs pedig a kód generátor mátrixának egy permutált és zavart verziója. Ennek a rendszernek az a nagy előnye, hogy az encapsulation rendkívül gyors (csak mátrix-vektor szorzás és hibahozzáadás), azonban a kulcsméretetek nagyok (több tízezer bit is lehet implementációtól függően).

A BIKE ezt a koncepciót modernizálja az MDPC (Moderate-Density Parity-Check) kódok felhasználásával, amelyek kompaktabb kulcsokat tesznek lehetővé az error-correcting kód-alapú titkosítás megtartása mellett.

2.3.2. MDPC kódok és az BIKE paraméterek

Az MDPC kódok olyan lineáris kódok, amelyekben a paritás-mátrix ritka (sparse), vagyis csupán néhány nem-nulla elemmel rendelkezik minden sorban. Formálisan, egy n hosszú MDPC kód a H paritásmátrixszal definiálható, ahol a H egy $m \times n$ mátrix, és az egyes soroknak csupán d_c nem-nulla elemei vannak (constant weight).

Adott egy hibavektor $\mathbf{e} \in \mathbb{F}_2^n$ (ahol az elemek 0 vagy 1), az $\mathbf{e}$ a kódhoz tartozik, ha $H \cdot \mathbf{e}^T = \mathbf{0}^T \pmod{2}$. Ha az $\mathbf{e}$ vektornak néhány nem-nulla bitje van (például t darab), akkor a $H \cdot \mathbf{e}^T$ nem lesz zérus, hanem az úgynevezett szindróma (syndrome).

Az MDPC dekódolási probléma lényege a következő: adott egy szindróma $\mathbf{s} = H \cdot \mathbf{e}^T$ és a H paritásmátrix, a feladat az eredeti hibavektor $\mathbf{e}$ helyreállítása rendszeres dekódolási algoritmusokkal. Ha a hibaszint és a kód paraméterei helyesen kerülnek megválasztásra, a dekódolás szinte garantált, ugyanakkor egy olyan támadó számára, aki a H teljes szerkezetét nem ismeri, a hibavektor visszaállítása gyakorlatilag lehetetlen.

A BIKE protokoll különböző szinteket kínál:

- **BIKE-L1:** 128 bites klasszikus biztonsági szint, $n = 12323$, $d = 71$, $t = 134$
- **BIKE-L3:** 192 bites klasszikus biztonsági szint, $n = 24659$, $d = 71$, $t = 199$
- **BIKE-L5:** 256 bites klasszikus biztonsági szint, $n = 40973$, $d = 71$, $t = 264$

Itt az n a kód hossza, a d az MDPC kód soros súlya (row weight), és a t az a maximális hiba-számlálás, amit a dekódoló még helyesen kezel. Az n és t értékek közvetlenül kapcsolódnak a biztonsági szinthez.

2.3.3. BIKE kulcsgenerálás és encapsulation

A BIKE protokoll szintén három fő algoritmusból áll: KeyGen, Encaps, és Decaps.

2.3.3.1. Kulcsgenerálás (KeyGen)

A BIKE kulcsgenerálása az MDPC kódok szerkezetéből indul:

1. Generáljunk egy véletlenszerű ritka mátrixot $H = [H_0, H_1]$, ahol a H_0 és H_1 az $m \times n$ alakú, és az egyes soroknak pontosan d nem-nulla elemük van (constant weight)
2. A H_0 sorainak az inverze (ha létezik) az L mátrix

3. A H -ből számítsuk ki az effektív paritás-mátrixot: $H' = L \cdot H = [I, H']$ (identity és egy H' mátrix)
4. A nyilvános kulcs: $pk = H'$ (vagy annak egy tömörített reprezentációja)
5. A privát kulcs: $sk = H$ (az eredeti paritás-mátrix és az L inverz mátrix információi)

2.3.3.2. Üzenet becsomagolása (Encaps)

A Kyber-hez hasonlóan, az encapsulation egy megosztott titkot hoz létre:

1. Sorsoljunk le egy véletlenszerű üzenet $m \in \{0, 1\}^n$ bits, de csak az első t bit legyen nem-nulla
2. A szindrómát számítsuk ki: $\mathbf{s} = H' \cdot \mathbf{m}^T \pmod{2}$
3. A kapszula: $c = \mathbf{s}$ (vagy egy tömörített verzió)
4. A megosztott titok: $K = H(\mathbf{m})$

2.3.3.3. Üzenet kicsomagolása (Decaps) és bit-flipping dekódolás

A decapsulation során az Alice ismeri az eredeti H mátrixot, így helyreállíthatja az üzenetet:

1. A szindrómából indulva ($\mathbf{s}$), a dekódoló algoritmus helyreállítja az eredeti hibavektort $\mathbf{e}'$
2. A helyreállított üzenet: $m' = \mathbf{e}'$
3. A megosztott titok: $K = H(m')$

A bit-flipping algoritmus az egyik legegyszerűbb és leggyakrabban használt MDPC dekódolási módszer. Az algoritmus iteratívan működik:

1. Inicializálás: az aktuális hibavektor $\mathbf{e}_0 = \mathbf{0}$
2. Iteráció $i = 1, 2, \dots, \text{max_iter}$:
 - (a) Számítsuk ki az aktuális szindrómát: $\mathbf{s}_i = \mathbf{s} + H \cdot \mathbf{e}_i^T \pmod{2}$
 - (b) Az j pozícióban lévő hiba-score: $\text{score}_j =$ az j -ik bit által befolyásolt szindróma bitek száma
 - (c) A legmagasabb pontszámú pozícióban fordítsuk az $\mathbf{e}_{i+1}$ bitet: $e_{i+1,j} = e_{i,j} \oplus 1$ ha $\text{score}_j \geq \text{threshold}$
 - (d) Ha az $\mathbf{s}_i = \mathbf{0}$, akkor sikeresen dekódoltunk
3. Ha az iterációs limit elérésre kerül, és még nem nullázta a szindrómát, akkor az algoritmus meghiúsul vagy egy feltételes választási módszert alkalmaz

A bit-flipping algoritmus hatékonysága az $\mathbf{m}$ hibavektor súlyától és a küszöb (threshold) választásától függ. A megfelelő paraméterek mellett az algoritmus szinte garantáltan helyreállítja a t nagyságrendű hibavektort. Az algoritmus nagy előnye az $O(n^2t)$ időbonyolultsága a H mátrix elég ritka volta miatt, amely lehetővé teszi az ilyen nagyméretű kódok praktikus alkalmazását.

2.3.4. Biztonsági tulajdonságok

A BIKE biztonsága az MDPC dekódolási probléma és az általános szindróma dekódolás NP-nehézségén alapul. Az ismeretes támadások közül az egyik legeredményesebb az úgynevezett *decoding failure attack*, amely azt próbálja kihasználni, hogy az MDPC dekódoló néha megghiúsulhat. Az újabban bevezetett kutatások azt mutatják, hogy a BIKE paraméterek a szükséges biztonsági szinteket nyújtják, ha az encapsulation transzformáció helyesen (például az Fujisaki-Okamoto transzformációval) kerül meghívásra az IND-CCA biztonság garantálásához.

A BIKE egy másik előnye az implementációs egyszerűsége. Az encapsulation és a kezdeti decoding csupán bitszintű XOR és összeadás műveleteket igényel, ami könnyen felgyorsítható szoftveres és hardveres eszközökkel. A Kyber-hez képest a BIKE nagyobb kulcsméreteket igényel, azonban az encapsulation részben gyorsabb lehet egyszerűbb eszközökön.

2.4. Oldalsávós támadások és biztonsági implementáció

A posztkvantum-kriptográfiai algoritmusok biztonságát nemcsak az elméleti matematikai alapjai, hanem a gyakorlati implementáció is meghatározza. Az oldalsávós támadások (side-channel attacks) olyan támadási módszerek, amelyek a fizikai implementáció melléktermékeit, mint az időzítési információ, energiafogyasztás vagy elektromágneses sugárzás kihasználják.

2.4.1. Időzítési támadások (Timing Attacks)

Az időzítési támadások azt használják ki, hogy az algoritmus futási ideje függhet a feldolgozott adatoktól. Például, ha az LWE dekódoló algoritmus gyorsabban fejeződik be bizonyos hibaértékek mellett, akkor egy támadó a futási időmérésből információkat szerezhet az eredeti titkos kulcsról.

A Kyber implementáció során különös figyelmet kell fordítani arra, hogy az összes operáció azonos időben fusson le, függetlenül az input értékeitől. Ez az úgynevezett konstant-time implementáció. Az NTT algoritmus például nem konstant-time természet szerint, ezért az implementáció során dummy operációkat kell beszúrni az időzítés kiegyenlítéséhez.

A BIKE esetén a bit-flipping dekódoló algoritmus szintén sérülékeny az időzítési támadásokra, mivel az iteráció száma az aktuális hibaminta sikerességétől függ. Az ajánlott megoldás a dekódolási iterációk számának rögzítése, még akkor is, ha az algoritmus korábban sikeresen helyreállított egy érvényes hibavektort.

2.4.2. Energiafogyasztási támadások (Power Analysis)

Az energiafogyasztási támadások az eszköz elektromos energiafogyasztásának megfigyelésén alapulnak. A különféle operációknak (összeadás, szorzás, memória-hozzáférés) különböző energiafogyasztása van, és ezek a különbségek felhasználhatók az algoritmus belső állapotával kapcsolatos információkra.

A CPA (Correlation Power Analysis) a legerősebb ismert energiafogyasztási támadás, amely statisztikai módszereket használ a megfigyelt energiajelek és az ismert algoritmuskimenet közötti korrelációk feltérképezésére. A Kyber és BIKE implementációknak maszkolási technikákat kell alkalmazniuk, ahol az összes kritikus adat logikai maszkokkal kombinálódik, hogy az energiajelek ne legyenek korrelálva a feldolgozandó érzékeny információkkal.

2.4.3. Hibabeviteli támadások (Fault Injection)

A hibabeviteli támadások során a támadó szándékosan hibákat vezet be az eszköz működésébe, például elektromágneses impulzus vagy lézer segítségével, és megfigyelik, hogy az algoritmus hogyan reagál. A hibázó számítások eredménye információt árulhat el az algoritmus titkos paramétereiről.

A Kyber és BIKE protokolloknak ellenállónak kell lenniük az ilyen támadásokkal szemben. Az egyik megközelítés a redundanciás számítások: az algoritmus több alkalommal fut, és az eredmények egyezéseit ellenőrzik. Ha egy hiba detektálódik, az algoritmus meghiúsul vagy kijelent egy hibát, helyett hogy helytelen eredményt adna.

2.5. IoT alkalmazások és integráció

Az Internet of Things (IoT) rendszerek gigantikus mértékben terjednek és a biztonsági megközelítés kritikus fontosságú. Az IoT eszközök között vannak erőforrás-korlátozott mikrokontrollerek (például ARM Cortex-M4), melyekre korlátozott memória (néhány kilóbájt RAM), korlátozott feldolgozási teljesítmény és alacsony energiabáziú működés jellemző.

2.5.1. Kyber IoT implementáció

A Kyber protokoll viszonylag kis kulcsméretei és jó párhuzamosíthatósága miatt alkalmas lehet az IoT alkalmazásokra. Az ARM Cortex-M4 processzor támogatja a SIMD szerű utasításokat, amelyekkel a Kyber szorzásokat jellegzetes módon fel lehet gyorsítani. Az OpenQuantumSafe és a liboqs-python nyílt forráskódú implementációi már biztosítanak optimalizált kódokat az ARM platformokra.

A Kyber512 paraméterek egy alap IoT eszközre teljesíthetők körülbelül 10-20 milliszekundumos encapsulation és decapsulation idővel, ami a legtöbb alkalmazás számára elegendő. Az energiafogyasztás egy „embedded” kontextusban körülbelül 10-50 millijoule egy teljes kulcscsere művelet során, ami elfogadható a legtöbb IoT alkalmazásban.

2.5.2. BIKE IoT implementáció

A BIKE protokoll nagyobb kulcsméreti és iteratív dekódolási folyamata kihívásokat jelentenek az erőforrás-korlátozott IoT eszközökön. Azonban az encapsulation művelet (amely a küldő oldalon történik) viszonylag gyors és energiahatékony, mivel csupán bit-szintű operációkra támaszkodik. Az erőforrás-korlátozott szándékolt decapsulation (a fogadó oldalon) azonban több időt és energiát igényelhet.

Egy Cortex-M4-en a BIKE-L1 paraméterekkel végzett decapsulation tipikusan 100-200 milliszekundum időt vesz igénybe, ami bizonyos alkalmazásokra még elfogadható, de az encapsulation-hoz képest lassabb.

2.5.3. Hibrid megközelítés az IoT-ben

Az ajánlott gyakorlat az IoT rendszerekben egy hibrid kulcscseremechanizmus, amely kombinálja az elliptikus görbe Diffie-Hellman (ECDH) kulcscserét egy posztkvantum-kriptográfiai protokollal (ML-KEM/Kyber vagy ML-KEM-BIKE). Ez azt jelenti, hogy az IoT eszköz két független kulcscsatornát létesít:

1. **Klasszikus csatorna:** ECDH kulcscsere, amely ma biztonságos klasszikus támadások ellen
2. **Posztkvantum csatorna:** ML-KEM (Kyber) kulcsencapsulation, amely biztonságos a jövőbeli kvantum-támadások ellen

A végső szimmetrikus kulcs a két csatorna kimenetének kombinációja:

$$K_{\text{final}} = H(K_{\text{ECDH}} \| K_{\text{ML-KEM}})$$

Ez a megközelítés biztosítja, hogy még akkor is, ha az egyik protokoll valahogy kompromittálódna, a rendszer biztonsága a másik algoritmus ereje által védett marad. Ezenkívül biztosítja, hogy az a mai klasszikus támadásokkal szemben megmarad az ECDH biztonság, valamint a jövőbeli kvantum-veszélyekkel szemben a Kyber nyújt védelmet.

2.5.4. Memória- és tárolási követelmények

Az IoT eszközök korlátozottak a háttértár méreteikben és memóriájukban. A Kyber512 nyilvános kulcsának mérete körülbelül 800 bájt, a privát kulcsé pedig 1632 bájt. Ez a legtöbb IoT eszközön közvetlenül Flash memóriában tárolható.

A BIKE-L1 paraméterek nagyobb kulcsokat igényelnek: körülbelül 1541 bájt nyilvános kulcs és 3104 bájt privát kulcs. Az erőforrás-korlátozott eszközökön ez szűkös lehet, és tömörítési technikákat (például az MDPC mátrix kompakt reprezentációja) lehet alkalmazni a méret csökkentésére.

Az encapsulation során létrehozott kapszula mérete szintén fontos az IoT kommunikációban. A Kyber512 kapszulájának mérete körülbelül 768 bájt, míg a BIKE-L1 kapszulájának mérete körülbelül 1541 bájt. Mivel az IoT hálózatokban az adatok átvitele költséges (energiafogyasztás és sávszélesség szempontjából), a kisebb kapszulaméretet előnyösebbek.

2.6. Kyber és BIKE összehasonlítása

A két protokoll közötti választás több tényezőtől függ: a biztonsági követelményektől, az alkalmazás teljesítmény-kritikussága és az elérhető hardver-támogatástól.

2.6.1. Biztonsági megfontolások

Mind a Kyber, mind a BIKE széles körben tanulmányozott és alapvető matematikai problémák NP-nehézségén alapulnak. Az LWE probléma és az MDPC dekódolás egyaránt nem rendelkezik ismert kvantumgyorsulással. A NIST standardizálási folyamatban mindkettő végig jól teljesített, de végül a Kyber lett az ajánlott KEM standard az ML-KEM megnevezéssel.

2.6.2. Teljesítmény- és méretjellemzők

A Kyber általában kisebb kulcsokat igényel az BIKE-hez képest. Például Kyber768 esetén a nyilvános kulcs mérete körülbelül 1184 bájt, míg BIKE-L3 esetén 1541 bájt. Az encapsulation sebesség hasonló, azonban a decapsulation során a Kyber nagy szorzások végzésére, míg a BIKE inkább bit-szintű operációkra támaszkodik. Ez azt jelenti, hogy a Kyber általában gyorsabb encapsulation és decapsulation műveleteket tesz lehetővé, különösen optimalizált implementációk esetén. Azonban a BIKE egyszerűbb műveletei miatt bizonyos környezetekben előnyösebbek lehetnek, különösen akkor, ha a kulcsméretet nem kritikusak.

2.6.3. Gyakorlati alkalmazások

Az IoT rendszerekben a Kyber előnyösebb a kisebb kulcsméretei és jó párhuzamosíthatósági lehetőségei miatt. Az olyan erőforrás-korlátozott eszközökön, mint az ARM Cortex-M

mikrokontrollerek, a Kyber szoftverszintű optimalizálása már megtörtént és elérhető nyílt forráskódú implementációja is.

A BIKE-nek nagyobb memóriefoglalata van, azonban az olyan speciális alkalmazásokban, ahol az encapsulation sebesség kritikus (például egy nagyon magas kódolási rátájú csatorna), szintén értékes lehet.

2.6.4. Hibrid megközelítés

A gyakorlatban az ajánlott megközelítés egy hibrid rendszer, amely klasszikus és poszt-kvantum kriptográfiai algoritmusokat kombinál. Például egy ECDH (Elliptic Curve Diffie-Hellman) kulcscsere mellett egy ML-KEM (Kyber) kulcscsatornát végeznek, és a végső szimmetrikus kulcs mindkettő kombinációja. Ez biztosítja, hogy még ha az egyik algoritmus kompromittálódna is (akár egy nem várt felfedezés, akár egy kvantum támadás miatt), a rendszer biztonsága megmarad a másik algoritmus ereje által.

3. fejezet

Mérési környezet

Jelen fejezet a kísérleti mérések elvégzéséhez kialakított hardver- és szoftverkörnyezetet mutatja be. A fejezet célja, hogy az eredmények reprodukálhatóságához szükséges összes technikai részletet rögzítse: az alkalmazott eszközök specifikációját, a szoftverkomponensek verzióit és a hálózati topológiát.

3.1. Hardverkörnyezet

3.1.1. Tesztplatform specifikációja

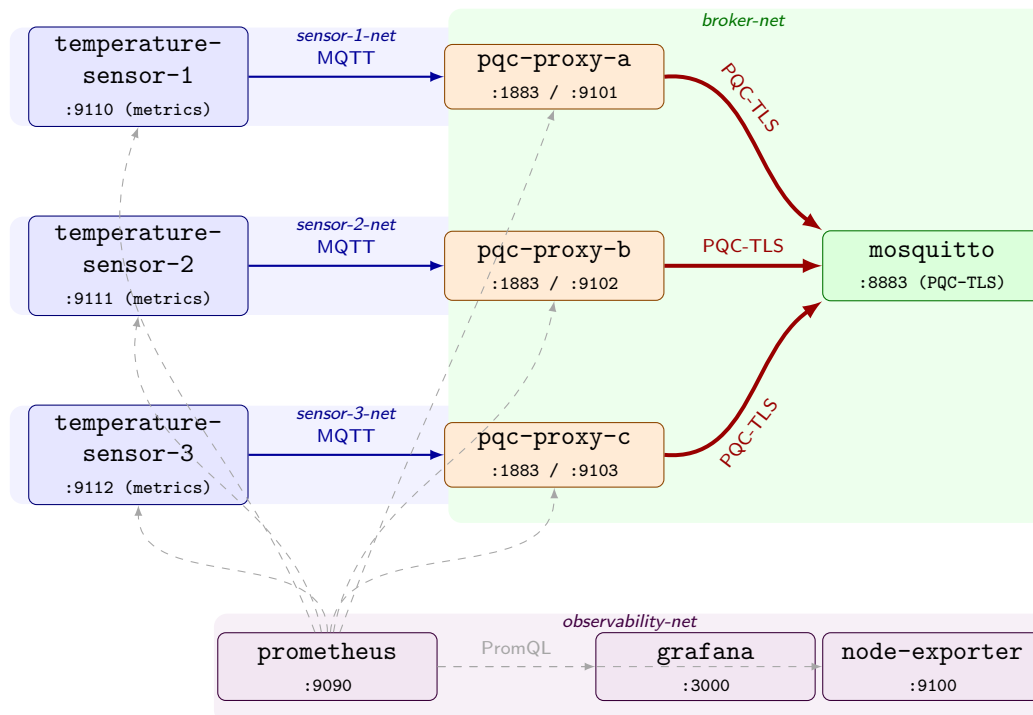
A mérések x86-64 architektúrájú asztali számítógépen futottak, Ubuntu 24.04 LTS operációs rendszer alatt. A konténerizált környezet lehetővé tette, hogy az egyes IoT-csomópontok és proxykomponensek erőforrásigényét a Docker Engine CPU-kvótamechanizmusán keresztül szoftveresen korlátozzam, így a fizikailag rendelkezésre álló teljesítmény helyett a korlátozott erőforrás-konfiguráció hatása kerülhetett mérés középpontjába. Ez a megközelítés jól modellezi azokat az ipari és beágyazott IoT szcenáriókat, amelyekben az eszközök számítási teljesítménye korlátozott.

3.1.2. Hálózati topológia

A labor hálózati rétege Docker bridge hálózatokból épül fel, amelyek izolált kommunikációs szegmenseket hoznak létre az egyes eszközcsoportok között. Az alkalmazott topológia a következő öt hálózatot tartalmazza:

- **broker-net**: nyilvános bridge hálózat, amelyen a Mosquitto MQTT broker és a három PQC proxy osztozik. Ezen folyik a PQC-TLS titkosított MQTT forgalom (8883/tcp).
- **sensor-1-net**, **sensor-2-net**, **sensor-3-net**: 3 internal Docker bridge hálózat, amelyek egyenként csak az adott IoT-csomópontot és a hozzá tartozó proxyt látják. Az IoT-csomópontok így nem érhetik el közvetlenül a brokert.
- **observability-net**: bridge hálózat, amelyen a Prometheus, a Grafana és az IoT-csomópontok metrikaexporterei kommunikálnak. Nincs közvetlen kapcsolata a **broker-net**-tel.

Ez a topológia reprodukálja azt az ipari mintát, ahol az IoT végpontok (**temperature-sensor-1/2/3**) egy dedikált biztonsági proxyn (jelen esetben egy PQC-képes stunnel sidecar-on, **pqc-proxy-a/b/c**) keresztül érik el az MQTT brokert, és így a PQC kézfogás kizárólag a proxy-broker szegmensen történik. A hálózati felépítést a 3.1. ábra szemlélteti.



3.1. ábra. A labor hálózati topológiája

3.1.3. Az adatfolyam és a titkosítás helye

Az alábbiakban lépésről lépésre bemutatom, hogy egy szenzorüzenet hogyan jut el az IoT-csomóponttól a brokerig, és pontosan hol lép be a kriptográfia.

1. **Üzenet előállítása az IoT-csomóponton.** Minden IoT-csomópont (temperature-sensor-1/2/3) egy Python 3.12 folyamat, amely 5 másodpercenként publikál egy szimulált hőmérsékleti értéket a `sensors/temperature` MQTT-topikra. Az üzenet JSON-formátumú:

```
{
  "sensor_id": "temperature-sensor-1",
  "temperature": 23.47,
  "timestamp": 1715500800.0
}
```

Ez az üzenet maga *nem tartalmaz semmilyen titkosítást*: az IoT-csomópont nem tölt be PQC-könyvtárat, és nem végez kriptográfiai műveletet.

2. **Titkosítatlan MQTT-szakasz: csomópont → proxy.** A PUBLISH csomag sima TCP-kapcsolaton (`sensor-N-net`, 1883/tcp) jut el az adott csomóponthoz dedikált `pqc-proxy-a/b/c` sidecar konténerébe. Ez az egy-egy izolált Docker bridge hálózaton fut, amelyre rajta kívül más konténer nem csatlakozik, így a titkosítás hiánya ezen a rövidített, helyi szegmensen nem jelent biztonsági kockázatot.
3. **PQC-TLS kézfogás: proxy ↔ broker.** A proxy indításakor a stunnel TLS 1.3 kézfogást kezdeményez a Mosquitto broker felé (8883/tcp, `broker-net`). A kézfogás KeyShare lépésében a konfigurált PQC algoritmus kerül alkalmazásra:
 - A **broker** (TLS-szerver) az `oqs-provider` segítségével generál egy KEM-kulcspárt, és a nyilvános kulcsot elküldi a proxynak.

- A **proxy** (TLS-kliens) a nyilvános kulccsal becsomagol egy véletlen titkot (*encapsulate*), majd a kapszulát visszaküldi a brokernek.
- Mindkét fél önállóan levezet egy azonos **munkamenetkulcsot** (*shared secret*) – a proxy a kapszulálás, a broker a visszafejtés (*decapsulate*) révén.

A munkamenetkulcsból a TLS 1.3 kulcslevezetési függvénye (HKDF) állítja elő a szimmetrikus forgalomvédelmi kulcsokat. *Ez az a lépés, ahol az ML-KEM, a BIKE vagy az X25519 algoritmus aktívan részt vesz.*

4. **Titkosított MQTT-forgalom: proxy → broker.** A kézfogás után a proxy a szenzortól érkező PUBLISH csomagot **AES-128-GCM** szimmetrikus titkosítással védi és továbbítja a brokernek. A TLS-rekordban utazó tartalom – beleértve a JSON hőmérsékleti adatot, a QoS 1 PUBLISH fejléct és a csomagazonosítót – a hálózaton kívülről olvashatatlan.
5. **Dekódolás a Mosquitto brokernél.** A Mosquitto broker TLS-rétege (stunnel szerver, szintén oqs-provider) a munkamenetkulccsal visszafejti a TLS-rekordot, és az így kapott sima PUBLISH csomagot az MQTT-protokollnak megfelelően dolgozza fel (visszaigazolás QoS 1 esetén: PUBACK).

Összefoglalva: a PQC algoritmus a munkamenetkulcs biztonságos megegyezésére (KEM) szolgál, nem közvetlenül az adat titkosítására. A szenzorüzenet tényleges védelme AES-128-GCM szimmetrikus titkosítással történik, amelynek kulcsát a PQC KEM teszi kvantumszámítógép ellen is ellenállóvá. A mérések során rögzített MQTT publish-ciklus készletet pontosan ezt a teljes kört – JSON előállítás, proxy-továbbítás, TLS-rekordküldés, PUBACK-visszatérés – foglalja magában.

3.1.4. Energiaellátás és mérőeszközök

A mérési összeállítás szoftveres metrikaexportereken alapult; dedikált hardveres energia-mérő eszköz (pl. INA219 áramszenzor) nem került alkalmazásra, így az energiafogyasztás közvetlen mérése nem történt meg. A processzorkihasználtságot a Python `psutil` könyvtár `cpu_percent()` hívásával becsülöm folyamatonként. A valódi energiafogyasztás meghatározásához jövőbeli munkában hardveres mérőeszköz integrálása javasolt.

3.2. Szoftverkörnyezet

3.2.1. Operációs rendszer és alapkönyvtárak

A hoszt operációs rendszer Ubuntu 24.04 LTS (linux/amd64). A kriptográfiai könyvtárakat igénylő komponensek (Mosquitto broker, PQC proxy) alapja a `pqc-base:latest` egyedi Docker image, amelyet a `base-image/Dockerfile` definiál: ez `ubuntu:24.04`-re épül, és forrásból fordítja be a `liboqs` és az `oqs-provider` könyvtárakat. Az IoT-csomópontok (`temperature-sensor-1/2/3`) `python:3.12-slim` alapképet használnak, mivel nem igényelnek PQC-könyvtárakat – a biztonságos TLS kézfogást a proxy sidecar végzi el helyettük. A `liboqs` és az `oqs-provider` fordításához szükséges build-függőségek: `build-essential`, `cmake`, `ninja-build`, `libssl-dev`, `python3-dev`, `git`, `curl`, `astyle` és `pkg-config`.

3.2.2. Kriptográfiai könyvtárak

A PQC-képes TLS stack két nyílt forráskódú komponensre épül, amelyeket forrásból kell fordítani, mivel a rendszercsomagok nem tartalmazzák őket:

- **liboqs** – az Open Quantum Safe projekt alap C-könyvtára, amely az összes mért PQC algoritmust implementálja. A build során a következő CMake opciók kerültek alkalmazásra: `-DOQS_DIST_BUILD=ON` (architektúrafüggetlen binárisok), `-DBUILD_SHARED_LIBS=ON` (megosztott könyvtárként), `-DOQS_USE_OPENSSL=ON` (OpenSSL-integrált hash-primitívek). A telepítési cél `/usr/local`, a könyvtárak `ldconfig` hívással kerülnek regisztrálásra.
- **oqs-provider** – az OpenSSL 3 Provider architektúráját megvalósító modul, amely a liboqs algoritmusait az OpenSSL kriptográfiai keretrendszerébe illeszti be. A fordítás a `liboqs_DIR=/usr/local/lib/cmake/liboqs` és `OPENSSL_MODULES_PATH=/usr/local/lib/openssl-modules` paraméterekkel történt. A kész `oqsprovider.so` dinamikus könyvtár az utóbbi könyvtárba kerül.

Az OpenSSL konfigurálása egy dedikált `/etc/ssl/oqs-openssl.cnf` fájlön keresztül történik, amelyet a `OPENSSL_CONF` környezeti változó mutat. Ez a konfiguráció aktiválja az `oqsprovider-t`, és a TLS 1.3 kulcscserecsoportok alapértelmezett listáját a következőképpen állítja be: `mlkem512`, `mlkem768`, `mlkem1024`, `X25519MLKEM768`, `bikel1`, `bikel3`, `bikel5`, `x25519`, `P-256`, `P-384`, `P-521`.

A TLS tanúsítványok hitelesítési algoritmusként ECDSA P-256-ot (prime256v1 görbe) alkalmaznak. A PQC algoritmus kizárólag a kulcscsere (KEM) fázisban jelenik meg a TLS 1.3 kézfogás során, a tanúsítványaláírásban nem. Ez összhangban van a hibrid migrációs ajánlásokkal, amelyek a tanúsítványlánc méretét klasszikus szinten tartják. A generált tanúsítványok mérete: CA-kulcs 227 bájt, CA-tanúsítvány 627 bájt, szerverkulcs 227 bájt, szerver-tanúsítvány 611 bájt (ECDSA P-256).

3.2.3. MQTT infrastruktúra

Az MQTT broker az Eclipse Mosquitto, amelyet a Ubuntu csomagkezelőjéből telepítettem (`mosquitto` + `mosquitto-clients` + `stunnel4`). A broker a loopback interfészen hallgat titkosítatlan MQTT-kapcsolatokra (1883/tcp, kizárólag 127.0.0.1-ről elérhető), míg a PQC-TLS lezárást a `stunnel` szerveroldali példánya végzi (8883/tcp, nyilvánosan elérhető a `broker-net-en` belül).

Az IoT-csomópontokat egy Python 3.12 alapú konténer valósítja meg (`python:3.12-slim` alapképpel). A csomópont a következő könyvtárakat használja: `paho-mqtt` 2.1.0 (MQTT 5 kliens), `prometheus_client` 0.21.0 (metrikaexport), `psutil` 6.1.0 (folyamat-szintű erőforrás-mérés). Az MQTT publikus intervallum 5 másodpercre lett beállítva (`PUBLISH_INTERVAL=5`), QoS 1 minőségi szinttel.

3.2.4. Konténerizáció és reprodukálhatóság

Az összes komponens Docker Compose v2 orkesztrációval indul, amelyet a `docker-compose.yml` definiál. Az algoritmus és a CPU-kvóta `.env` fájlból propagál minden konténerhez, így az algoritmusváltáshoz nem szükséges az image-ek újraépítése, csupán elegendő a PQC és ALG változó módosítása, majd a konténerek újraindítása.

A megfigyelési réteget négy komponens alkotja: Prometheus (`prom/prometheus:latest`), Grafana (`grafana/grafana:latest`), cAdvisor (`gcr.io/cadvisor/cadvisor:latest`) és node-exporter (`prom/node-exporter:latest`). A Prometheus – hogy az összes célpontot elérhesse – egyszerre tagja az `observability-net`, a `broker-net` és mindhárom `sensor-*-net` hálózatnak, és 10 másodpercenként gyűjt adatot a következő végpontokról:

- IoT-csomópontok (`temperature-sensor-{1,2,3}:9100`) – MQTT publish-ciklus késleltetés, üzenetszám, erőforrás-metrikák;

- PQC proxyk (`pqc-proxy-{a,b,c}:9101`) – TLS kézfogás késleltetése (`handshake_latency_ms`), induláskor mérve és statikusan tárolva;
- cAdvisor (`cadvisor:8080`) – konténer-szintű CPU-, memória- és hálózati metrikák valós időben;
- node-exporter (`node-exporter:9100`) – hoszt-szintű rendszermetrikák.

A Grafana dashboardja automatikusan konfigurált (provisioning) a `grafana/provisioning/` könyvtárból.

3.2.5. A Prometheus és a Grafana szerepe a mérési infrastruktúrában

A kísérleti hálózatban a megfigyelési réteg nem opcionális kiegészítő, hanem a mérési módszer szerves része: az összes kvantitatív eredmény – a késleltetés-hisztogramok, a memóriahasználat, az újracsatlakozásszám – kizárólag a Prometheus által gyűjtött metrikákból származik.

Prometheus szükségessége. Az IoT-csomópontok Python-kódja a `prometheus_client` könyvtáron keresztül exportálja a mért értékeket: az MQTT publish-ciklus késleltetését hisztogramként, az üzenetszámot számlálóként, az RSS-memóriát mérőszámként. Ezek az értékek az egyes konténerek belső memóriájában keletkeznek. A tartós tároláshoz és összesítéshez egy külső gyűjtőre van szükség. A Prometheus pull-alapú modellje 10 másodpercenként lekérdezi az összes végpontot, és idősoros adatbázisában tárolja az eredményeket. A kísérlet végén a mért adatok ebből az adatbázisból kerülnek exportálásra a `PromQL /api/v1/query` végpontra keresztül. Prometheus nélkül a konténerek leállásakor minden metrika elveszne.

Grafana szükségessége. A kísérletek futása közben elengedhetetlen a valós idejű visszajelzés: látni kell, hogy az IoT-csomópontok ténylegesen publikálnak-e, a PQC proxy sikeresen épít-e TLS-kapcsolatot, és a késleltetések a várt tartományban maradnak-e. A Grafana a Prometheus adatbázisát adatforrásként használva élő dashboardon jeleníti meg ezeket az értékeket, lehetővé téve a kísérlet közbeni hibakeresést – pl. ha egy csomópont nem csatlakozik, a publish-ráta azonnal nullára esik a dashboardon. Az `observability-net` hálózati szegmens kizárólag ennek a kommunikációnak ad otthont: Prometheus ↔ Grafana, és Prometheus ↔ cAdvisor/node-exporter között.

cAdvisor és node-exporter szerepe. A cAdvisor a Docker Engine API-ján keresztül konténer-szintű CPU- és memóriafelhasználást mér, a node-exporter pedig a hoszt operációs rendszer szintjén biztosít rendszermetrikákat. Ezek kiegészítik az alkalmazás-szintű metrikákat. Amíg az IoT-csomópont saját RSS-értékét `psutil`-al méri, a cAdvisor a Docker cgroup-korláttal összevethető külső nézőpontot nyújt, ami a CPU-kvóta hatásának ellenőrzésekor kulcsfontosságú.

3.3. A mérési környezet összefoglalása

A 3.1. táblázat összefoglalja a mérési labor főbb szoftverkomponenseit, azok verzióit és szerepeiket. A komponensek köre lefedi a teljes mérési stacket: a konténerizációs rétegtől a PQC könyvtárakon és az MQTT stack-en át a megfigyelési infrastruktúráig (Prometheus, Grafana, cAdvisor, node-exporter).

A labor legfontosabb eltérése egy valódi IoT telepítéstől az, hogy a mérési platform x86-64 architektúrán fut Docker CPU-kvótával emulált korlátok mellett, nem natív ARM mikrovezérlőn. Az ebből fakadó különbségek: az x86-64 hardveres AES-NI és AVX2 utasítások gyorsíthatják a szimmetrikus primitíveket, míg egy ARM Cortex-M4 vagy Cortex-A53 más utasításkészleten (NEON, nincs AVX2) hajtja végre ugyanazokat a műveleteket.

3.1. táblázat. A mérési környezet főbb komponensei

Komponens	Verzió / konfiguráció	Szerepe
Host OS	Ubuntu 24.04 LTS (x86-64)	Hoszt platform
Docker Engine	≥ 24 , Compose v2	Konténerizáció
pqc-base image	ubuntu:24.04 + liboqs + oqs-provider	Broker és proxy alapja
IoT-node image	python:3.12-slim	Szenzor csomópontok alapja
liboqs	main ág (2026. jún.)	PQC algoritmusok
oqs-provider	main ág (2026. jún.)	OpenSSL integráció
OpenSSL	3.x (rendszer, Ubuntu 24.04)	TLS stack
stunnel4	apt (Ubuntu 24.04)	TLS proxy/sidecar
Eclipse Mosquitto	apt (Ubuntu 24.04)	MQTT broker
paho-mqtt	2.1.0	MQTT kliens
prometheus_client	0.21.0	Metrikaexport
psutil	6.1.0	Erőforrás-mérés
Prometheus	prom/prometheus:latest	Metrikagyűjtés
Grafana	grafana/grafana:latest	Vizualizáció
cAdvisor	gcr.io/cadvisor:latest	Konténer-szintű metrikák
node-exporter	prom/node-exporter:latest	Hoszt-szintű metrikák

A mért eredmények így az algoritmusok relatív teljesítménykülönbségét és a CPU-korlát hatását rögzítik megbízhatóan, az abszolút futási idők azonban egy adott ARM-eszköz esetén eltérhetnek.

4. fejezet

Mérési metodológia

Jelen fejezet leírja a kísérleti mérések tervezési elveit, a mért paramétereket, az alkalmazott mérési módszereket és a statisztikai kiértékelés módját. A célkitűzés egy olyan ismételhető, automatizált kísérletmátrix kialakítása volt, amely lehetővé teszi a nyolc vizsgált algoritmus – négy ML-KEM-variáns (ML-KEM-512, ML-KEM-768, ML-KEM-1024, illetve a hibrid X25519MLKEM768), három BIKE-variáns (BIKE-L1, BIKE-L3, BIKE-L5) és a klasszikus X25519 referencia – rendszerszintű összehasonlítását változó CPU-erőforrás-korlátok mellett.

4.1. A kísérlet tervezési elvei

4.1.1. Szimulált erőforrás-korlátok

A dolgozat fő vizsgálati kérdése az, hogy a PQC algoritmusok milyen terhelést rónak az IoT-csomópontokra és a biztonsági proxykomponensekre erőforrás-szűkös körülmények között. Valódi heterogén hardverparkot összehasonlíthatóan mérni nehéz, ezért a labor Docker Engine CPU-kvótamechanizmusát alkalmazza: a `--cpus` paraméter meghatározza, hogy egy konténer az egész CPU-időszlet mekkora hányadát kaphatja meg. Az öt alkalmazott értéket az `scripts/run_experiment.sh` szkript Passmark-benchmarkon alapuló platform-megfeleltetéssel rendeli valós IoT-eszközosztályokhoz:

- **0,026 mag** – erősen korlátozott beágyazott eszköz, pl. ESP32 (Xtensa LX7, 240 MHz, ≈ 57 Passmark-pont).
- **0,12 mag** – belépő szintű Linux-alapú csomópont, pl. Raspberry Pi Zero 2W (Cortex-A53, 1 GHz, ≈ 220 Passmark-pont).
- **0,25 mag** – közepes kategóriájú peremeszköz, pl. Raspberry Pi 3B+ (Cortex-A53, 1,4 GHz, ≈ 450 Passmark-pont).
- **0,5 mag** – erős peremeszköz, pl. Raspberry Pi 4B (Cortex-A72, 1,5 GHz, ≈ 950 Passmark-pont).
- **1,0 mag** – referenciakonfiguráció; ipari IoT átjáró, közelítőleg Intel Atom szintű teljesítmény.

4.1.2. Kísérletmátrix és ismétlések

A teljes kísérletmátrixot a 8 algoritmus és 5 CPU-korlát kombinációja alkotja. Minden kombinációhoz **egy futtatás** tartozik, amelynek időtartama **300 másodperc**. Ez az időtartam elegendő ahhoz, hogy:

1. az MQTT csomópontok stabil kapcsolatot létesítsenek,
2. a Prometheus legalább 30 adatpontot gyűjtsön (10 s intervallum),
3. az MQTT latencia-hisztogram statisztikailag értékelhető legyen (≈ 60 üzenet csomópontonként, 5 s-es publish-intervallum mellett).

4.2. A mért paraméterek

4.2.1. TLS kézfogás késleltetése

A `handshake_latency_ms` Prometheus gauge minden PQC proxy konténer indításakor kerül mérésre. A proxy entypointja három egymást követő `openssl s_client` hívást hajt végre a Mosquitto broker 8883-as portjára, a konfigurált PQC kulcscsere-csoportot (`-groups $PQC_ALG`) és TLS 1.3-t (`-tls1_3`) megadva. Az eltelt falióraidőt nanoszekundum-pontosságú `date +%s%N` hívásokkal méri, majd a három mérés számtani átlagát exportálja Prometheus-metrikaként. Mivel a mérés konténer-induláskor egyszer fut le, az itt kapott érték az algoritmus hideg (cold-start) TLS kézfogási késleltetését rögzíti.

4.2.2. MQTT publish-ciklus késleltetése

Az MQTT publish-ciklus késleltetése az IoT-csomópontban méri azt az időt, amely az `mqttclient.publish()` hívása és a hozzá tartozó `on_publish` visszahívás aktiválása között telik el. Ez az idő magában foglalja:

- a QoS 1 üzenet elküldetési és PUBACK fogadási ciklusát a kliens és a proxy között,
- a proxy MQTT-forgalom stunnel-en keresztül továbbítását,
- az esetleges hálózati pufferhézagokat.

Az értékeket a `mqtt_publish_latency_ms` Prometheus hisztogram tárolja, amelynek vödörhatárai: 1, 2, 5, 10, 25, 50, 100, 250, 500, 1000, 2500, 5000, 15000, 30000 ms. A felső határok (15 000 és 30 000 ms) szándékosan illeszkednek a PUBACK-reaper timeout-jához (ld. 4.2.4. pont), hogy a sikertelen kézbesítések is megjelenjenek a hisztogramban. A kiértékelés során a p_{50} , p_{95} és p_{99} percentiliseket használom, amelyeket a `histogram_quantile()` PromQL-függvénnyel számítom az adott futtatás teljes időablakára.

4.2.3. Üzenetátviteli sebesség

Az `mqtt_messages_total` számláló rögzíti az MQTT-csomópont által elindított `publish()` hívások számát. Az üzenetküldési sebesség (msg/s) a futtatási ablakra vetített ráta, amelyet a Prometheus `rate()` függvényével számítom. Az elvárt érték 5 s-es intervallumnál 0,2 msg/s; az ettől való eltérés megbízhatatlan kapcsolatra vagy MQTT újracsatlakozásokra utal.

4.2.4. PUBACK-visszaigazolás és időtúllépés-nyomon követés

A QoS 1 MQTT kézbesítési megbízhatóságának mérésére az IoT-csomópont három számlálót vezet:

- `mqtt_publish_acked_total` – az `on_publish` visszahívásban sikeresen megérkezett PUBACK-visszaigazolások száma.

- `mqtt_publish_timeout_total` – azok a küldések, amelyekhez 30 másodpercen belül (`PENDING_TIMEOUT_S=30`) nem érkezett PUBACK.
- `mqtt_reconnect_total` – a broker felőli megszakítások száma, amelyek újracsatlakozást váltanak ki.

A PUBACK-reaper egy háttérrel, amely 5 másodpercenként ellenőrzi a függőben lévő publikálások listáját. Ha egy `mid` bejegyzés 30 s-nél régebbi, a tényleges eltelt idő bekerül a `mqtt_publish_latency_ms` hisztogramba, és a `mqtt_publish_timeout_total` számláló növekszik. Ez a mechanizmus különösen fontos a BIKE kísérletekben: stunnel-kapcsolati hiba esetén a QoS 1 PUBACK-ok nem érkeznek meg, és a reaper nélkül az üzenetek csendben eltűnnének a hisztogramból – az instabilitás láthatatlan maradna. Az ACK-arány (`ack_ratio = mqtt_publish_acked_total/mqtt_messages_total`) az algoritmus kézbesítési megbízhatóságának összefoglaló mutatója.

4.2.5. Folyamatszintű memóriafoglalás

A `node_memory_rss_bytes` gauge az IoT-csomópont Python-folyamatának pillanatnyi RSS (Resident Set Size) memóriahasználatát rögzíti, amelyet a `psutil.memory_info().rss` hívás ad vissza. Az átlagos RSS-t az `avg_over_time()` PromQL-aggregátorral számítom a futtatási ablakra. Ez a metrika jelzi, hogy a PQC-könyvtár betöltése és a folyamatos MQTT-forgalom milyen statikus memóriaterhelést jelent.

4.2.6. Processzorkihasználtság

A `node_cpu_usage_percent` gauge a `psutil.cpu_percent(interval=None)` hívást alkalmazza, amely a legutóbbi mérés óta eltelt CPU-időt adja vissza százalékban. Megjegyzendő, hogy az első hívás után `interval=None` használatakor a `psutil` az előző `cpu_percent()` hívás óta mért kumulatív értéket ad. Ha az inicializáló hívás és a tényleges mérési hívás között nagyon rövid az idő, a visszaadott érték nulla lehet. Ez az ismert mérési korlát az eredmények értelmezésekor figyelembe veendő.

4.3. Adatgyűjtési és futtatási eljárás

4.3.1. Automatizált kísérletfuttatás

A kísérletmátrix futtatását a `scripts/run_experiment.sh` szkript vezérli. A szkript algoritmuson és CPU-limiten végigiterálva minden kombinációra elvégzi a következő lépéseket:

1. **Tanúsítványgenerálás:** a `scripts/generate_certs.sh` friss ECDSA P-256 CA-és szerver-tanúsítványt állít elő a `certs/` könyvtárba. Ez minden futtatás előtt lefut, biztosítva, hogy a tanúsítványok nem járnak le és nem szennyeznek egymást.
2. **Konténerek újraindítása:** `docker compose up -d --force-recreate` paranccsal minden konténer friss állapotból indul, a `PQC_ALG`, `NODE_CPUS` és `PROXY_CPUS` értékekkel felülírva.
3. **Várakozás:** 300 másodperc telik el, amely alatt a Prometheus adatokat gyűjt, az IoT-csomópontok MQTT üzeneteket küldenek.
4. **Prometheus TSDB-pillanatkép:** a `/api/v1/admin/tsdb/snapshot` végpont meghívásával a Prometheus idősoradatai snapshotba kerülnek, biztosítva az offline utólagos elemzhetőséget.

5. **Metaadatok mentése:** algoritmus, CPU-korlát, időtartam és a snapshot azonosítója JSON-fájlba kerül (`experiment-results/$RUN_ID.json`).

4.3.2. Metrikák offline exportálása

A mérések befejeztével a `scripts/export_metrics.py` szkript az egyes futtatások Prometheus-snapshotjait lekérdezi a Prometheus HTTP API-n keresztül, és aggregált eredménytáblázatot készít. A lekérdezések a futtatás teljes időablakára vonatkoznak, és a három IoT-csomópontra (`temperature-sensor-1`, `temperature-sensor-2`, `temperature-sensor-3`) elkülönítetten közlik a percentiliseket, az ACK-arányt, a timeout- és újracsatlakozás-számokat, az RSS-memóriát és az üzenetküldési rátát. Az összesített exportfájl (`experiment-results/metrics_export.txt`) az eredmények fejezetének elsődleges adatforrása.

4.3.3. Statisztikai módszer

A teljes kísérletmátrix $8 \text{ algoritmus} \times 5 \text{ CPU-szint} = 40$ konfigurációt tartalmaz. Minden (algoritmus, CPU-korlát) párhoz egy 300 s-os futtatás tartozik, amelyben három IoT-csomópont párhuzamosan üzemel, így kombinációként három megfigyelés áll rendelkezésre. A mérés táblázataiban az átlag a három csomópont értékeinek számtani közepe. Ahol a csomópontok közötti szórás a középértékhez képest nagy (pl. p_{99} értékeknél alacsony CPU-korlát esetén), ott ezt külön jelzem.

4.4. A mérési megközelítés korlátai

A kialakított módszertan több szempontból is eltér egy ideális hardveres IoT mérési összeállítástól:

- **Emulált erőforráskorlát:** a Docker CPU-kvóta a Linux CFS ütemező időszelet-korlátján alapul, nem tényleges órajel-csökkentésen. A valódi Cortex-M4 más gyorsítótár-hierarchiával, más memória-sávszélességgel rendelkezik és nincs AVX2/AES-NI utasításkészlete, ami az abszolút futási idők tekintetében eltérést okozhat.
- **Hideg kézfogásmérés:** a `handshake_latency_ms` a proxy indításakor kerül mérésre, nem steady-state forgalom közben. A valódi IoT rendszerekben az újracsatlakozások és a session-resumption (TLS session ticket) eltérő képet adna.
- **CPU-kihasználtság mérési artefaktum:** a `psutil.cpu_percent(interval=None)` hívás első meghívásakor nulla értéket adhat vissza, ezért a CPU-adatok a mérési ciklus egészére nem megbízhatóak. Ez az értékelés során figyelembe kell venni.
- **Hálózati latencia:** a mérések Docker belső bridge hálózaton zajlanak, amely submilliszekundumos késleltetéssel rendelkezik. Valódi IoT hálózatokban (LoRaWAN, NB-IoT, Wi-Fi) a hálózati késleltetés dominálhat, elfedve az algoritmusok közötti különbségeket.

Ezek a korlátok az eredmények értelmezésekor és a javaslatok fejezetben figyelembe kerülnek.

5. fejezet

Mérések

Jelen fejezet a kísérleti mérések végrehajtásának eredményeit tartalmazza. Összesen 40 futtatás történt ($8 \text{ algoritmus} \times 5 \text{ CPU-korlát} \times 1 \text{ futtatás}$), mindegyik 300 másodpercig tartott, és mindegyikben három IoT-csomópont (`temperature-sensor-1`, `temperature-sensor-2`, `temperature-sensor-3`) párhuzamosan üzemelt. Az eredmények elsődleges forrása a `metrics_export.txt` összefoglaló fájl és a Prometheus TSDB-pillanatképek.

5.1. MQTT publish-ciklus késleltetése – X25519 (klasszikus referencia)

Az X25519 (klasszikus Diffie–Hellman Curve25519) referenciamérések eredményei az 5.1. táblázatban láthatók.

5.1. táblázat. X25519 MQTT publish-ciklus késleltetés (ms) CPU-korlát szerint

CPU-korlát	p_{50} (ms)	p_{95} (ms)	p_{99} (ms)
0,026 mag (ESP32)	1,68	165,9	452,2
0,12 mag (RPi Zero 2W)	1,56	49,0	86,8
0,25 mag (RPi 3B+)	1,53	4,2	75,6
0,5 mag (RPi 4B)	1,53	2,7	29,2
1,0 mag (Ipari átjáró)	1,50	2,2	7,5

Az X25519-nél markáns küszöbhatás figyelhető meg a 0,12 és 0,25 mag között. A medián értéke ($\approx 1,50\text{--}1,68$ ms) stabil, a 0,25 mag feletti p_{95} ($\approx 2,2\text{--}4,2$ ms) alacsony; a publish-ciklus latenciáját a helyi Docker-hálózati késleltetés és a QoS 1 PUBACK-ciklus dominálja, nem az algoritmus számítási igénye. Ez a klasszikus referenciamérés képezi az összehasonlítás alapját a következő fejezetekben vizsgált PQC algoritmusokhoz.

Az üzenetküldési sebesség 0,026 magon $\approx 0,175$ msg/s, 0,12 mag felett minden konfigurációban $\approx 0,20$ msg/s; az összes csomópont stabil kapcsolatot tartott fenn.

5.2. ML-KEM algoritmusok MQTT mérési eredményei

A CRYSTALS-Kyber algoritmuscsalád (NIST FIPS 203, ML-KEM) három biztonsági szinten – ML-KEM-512 (NIST I.), ML-KEM-768 (NIST II.) és ML-KEM-1024 (NIST III.) – került vizsgálatra. Mindegyik variáns esetén minden CPU-szinten sikeresen rögzíthetők voltak az MQTT publish-ciklus késleltetési percentilisek. A csomópontok stabil QoS 1 kapcsolatot tartottak fenn, az `ack_ratio` minden futtatásban 1,000.

5.2.1. ML-KEM-512

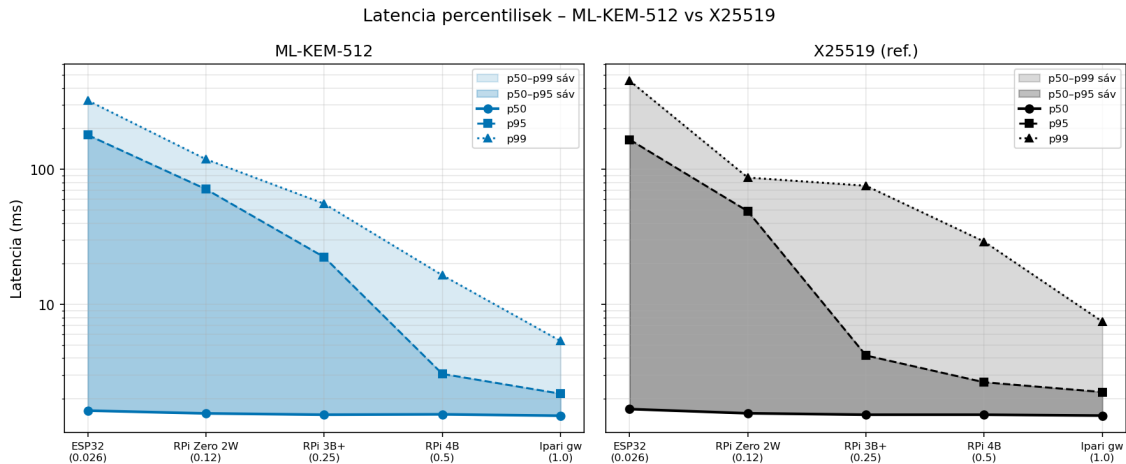
Az ML-KEM-512 a legalacsonyabb biztonsági szintű ML-KEM variáns (NIST I., AES-128-ekvivalens). Az 5.2. táblázat az MQTT publish-ciklus késleltetési percentilisek három csomópontonra átlagolt értékeit mutatja CPU-korlát szerint.

5.2. táblázat. ML-KEM-512 MQTT publish-ciklus késleltetés (ms) CPU-korlát szerint

CPU-korlát	p_{50} (ms)	p_{95} (ms)	p_{99} (ms)
0,026 mag (ESP32)	1,64	179,3	323,1
0,12 mag (RPi Zero 2W)	1,56	71,3	118,7
0,25 mag (RPi 3B+)	1,53	22,4	55,6
0,5 mag (RPi 4B)	1,54	3,1	16,4
1,0 mag (Ipari átjáró)	1,50	2,2	5,4

A p_{50} medián értéke minden CPU-szinten 1,50–1,64 ms között mozog, szintén CPU-független. A p_{95} és p_{99} percentilisek 0,026 magon (179,3 ms és 323,1 ms) magasak, jelezve az ESP32 szintű CPU-szűkösség hatását a stunnel proxyn. A 0,25 magon az egyes csomópontok között széles szórás figyelhető meg: egyes csomópontoknál $p_{95} < 3$ ms, míg másoknál 63 ms feletti értéket mértem, ami arra utal, hogy a küszöbhatár pontosan ennél a CPU-szintnél van. A 0,5 mag és 1,0 mag szinteken a p_{95} már stabilan ≈ 2 –3 ms.

A p_{50} – p_{99} percentilissávok szélességét – azaz a tipikus és a legrosszabb eset közötti különbséget – az 5.1. ábra szemlélteti az ML-KEM-512 és az X25519 referencia adataira. A minta az összes vizsgált PQC variánsra jellemző.



5.1. ábra. MQTT publish-ciklus késleltetési percentilissávok – ML-KEM-512 és X25519

Az 5.1. ábrán jól látható, hogy a p_{50} medián értéke (folytonos vonal) mindkét algoritmusnál minden CPU-szinten közel konstans, $\approx 1,5$ ms körül marad – a sáv alsó határa alig változik. Az árnyalt terület (a p_{50} és p_{99} közötti tartomány) ezzel szemben erősen CPU-függő: ESP32 szinten a sáv több mint két nagyságrendet ölel fel ($\approx 1,5$ ms-tól ≈ 330 ms-ig ML-KEM-512 esetén, illetve ≈ 450 ms-ig X25519 esetén), míg RPi 3B+ szinttől (0,25 mag) a sáv szinte összezsúrol. Ez a vizuális megjelenítés megerősíti a küszöbhatás meglétét: 0,25 mag felett a legrosszabb eset és a tipikus eset közötti különbség elhanyagolható.

5.2.2. ML-KEM-768

Az ML-KEM-768 (Kyber-768, NIST FIPS 203) algoritmus esetén minden CPU-szinten mind a három csomópontonál sikeresen rögzíthetők voltak a MQTT publish-ciklus késleltetési percentilisek. Az eredményeket a 5.3 táblázat foglalja össze. Az értékek egy futtatás három csomópontja (összesen három megfigyelés) számtani átlagai.

5.3. táblázat. ML-KEM-768 MQTT publish-ciklus késleltetés (ms) CPU-korlát szerint

CPU-korlát	p_{50} (ms)	p_{95} (ms)	p_{99} (ms)
0,026 mag (ESP32)	1,65	235,4	482,6
0,12 mag (RPi Zero 2W)	1,53	56,8	91,4
0,25 mag (RPi 3B+)	1,54	4,1	80,9
0,5 mag (RPi 4B)	1,52	2,3	8,6
1,0 mag (Ipari átjáró)	1,52	2,2	6,6

A p_{50} medián értéke minden CPU-szinten közel azonos, 1,52–1,65 ms között mozog, jelezve, hogy a tipikus (konfliktus-mentes) MQTT tranzakció nem érzékeny a CPU-korlát változására. A p_{95} és p_{99} percentilisek azonban erősen CPU-függők: 0,026 magon (ESP32 szint) a p_{95} közel 235 ms, a p_{99} meghaladja a 482 ms-ot; ezek ritka, de nagy késleltetésű TLS-kézfogási torlódásokhoz köthetők az erősen korlátozott proxy konténerben. A 0,12 magra (RPi Zero 2W szint) való emelkedéssel a p_{95} 57 ms-ra csökken; a 0,25 mag (RPi 3B+ szint) felett pedig markáns küszöbhatás figyelhető meg: a p_{95} mindössze 4,1 ms-ra esik vissza, ami jelzi, hogy a proxy elegendő CPU-időt kap a stunnel-kapcsolat stabil fenntartásához.

Az üzenetküldési sebesség 0,026 magon $\approx 0,168$ msg/s (kisebb torlódások miatt), 0,12 mag felett azonban minden konfigurációban eléri a $\approx 0,20$ msg/s elméleti maximumot, ami pontosan megfelel az 5 s-es közzétételi intervallumnak.

5.2.3. ML-KEM-1024

Az ML-KEM-1024 a legmagasabb biztonsági szintű ML-KEM variáns (NIST III., AES-256-ekvivalens). A 5.4. táblázat a három csomópontra átlagolt késleltetési percentiliseket tartalmazza.

5.4. táblázat. ML-KEM-1024 MQTT publish-ciklus késleltetés (ms) CPU-korlát szerint

CPU-korlát	p_{50} (ms)	p_{95} (ms)	p_{99} (ms)
0,026 mag (ESP32)	1,65	149,7	280,4
0,12 mag (RPi Zero 2W)	1,54	60,6	92,1
0,25 mag (RPi 3B+)	1,54	3,3	71,2
0,5 mag (RPi 4B)	1,53	3,2	50,2
1,0 mag (Ipari átjáró)	1,50	2,0	9,3

Az ML-KEM-1024 esetén a p_{95} küszöbhatása a 0,12–0,25 mag átmenetben van, az ML-KEM-768-cal megegyező módon. A p_{99} a 0,25 és 0,5 magon (≈ 71 ms, illetve ≈ 50 ms) valamivel magasabb az ML-KEM-768-éhoz (≈ 81 ms, illetve ≈ 9 ms) képest, ami a nagyobb kulcsméretnek (1568 B nyilvános kulcs, 1568 B kapszula) miatti marginálisan magasabb TLS-rekordkezelési overhead-del magyarázható. 1,0 magon a p_{99} 9,3 ms-ra csökken, ami az ML-KEM-768 6,6 ms-ához hasonló, és gyakorlatilag egyenértékű teljesítményt jelent.

5.3. X25519+ML-KEM-768 (hibrid)

Az X25519+ML-KEM-768 hibrid konfiguráció az X25519 klasszikus és az ML-KEM-768 posztkvantum kulcscsere párhuzamos alkalmazása egyazon TLS kézfogáson belül, biztosítva, hogy az összeköttetés klasszikus és kvantum szempontból is biztonságos maradjon. A 5.5. táblázat a három csomópontra átlagolt késleltetési értékeket mutatja.

5.5. táblázat. X25519+ML-KEM-768 hibrid MQTT publish-ciklus késleltetés (ms) CPU-korlát szerint

CPU-korlát	p_{50} (ms)	p_{95} (ms)	p_{99} (ms)
0,026 mag (ESP32)	1,67	170,7	285,6
0,12 mag (RPi Zero 2W)	1,52	35,8	88,9
0,25 mag (RPi 3B+)	1,52	2,1	71,2
0,5 mag (RPi 4B)	1,52	2,4	18,7
1,0 mag (Ipari átjáró)	1,49	2,0	8,5

A hibrid variáns viselkedése az ML-KEM-768-éhoz nagymértékben hasonló: a p_{95} küszöbhatása szintén a 0,12–0,25 mag sávban van. A 0,12 magon a $p_{95} \approx 35,8$ ms, ami alacsonyabb, mint az ML-KEM-768 56,8 ms-os értéke; ez az egyes csomópontok közötti mérési szórásból ered (az egyik szenzor már 0,12 magon is < 5 ms-os p_{95} -öt produkált). A p_{99} a 0,25 magon ≈ 71 ms, 0,5 magon ≈ 19 ms, 1,0 magon ≈ 9 ms. A hibrid megközelítés tehát nem jelent érzékelhető latencianövekedést az önálló ML-KEM-768-hoz képest, miközben klasszikus biztonsági garanciát is nyújt a posztkvantum migráció átmeneti időszakában.

5.4. BIKE algoritmusok mérési eredményei

A BIKE (Bit Flipping Key Encapsulation) algoritmuscsalád három biztonsági szinten – BIKE-L1 (NIST I.), BIKE-L3 (NIST III.) és BIKE-L5 (NIST V.) – vizsgálatra került. Mindhárom variáns esetén azonos jelenség figyelhető meg: a MQTT publish-ciklus késleltetési hisztogram minden CPU-korlát esetén kizárólag 30 000 ms értékeket tartalmaz, és az újracsatlakozási szám minden futtatásban magas.

5.4.1. BIKE-L1

A BIKE-L1 (Bit Flipping Key Encapsulation, Level 1) futtatásai meglepő eredményt hoztak. A `metrics_export.txt` összesítőben a BIKE-L1 csomópontok MQTT publish-ciklus késleltetési hisztogramja *minden CPU-korlát esetén kizárólag 30 000 ms értékeket* tartalmaz; a p_{50} , p_{95} és p_{99} percentilisek egyaránt erre az értékre illeszkednek, ahogyan azt a 5.6. és 5.7. táblázat CPU-korlát szerinti bontásban részletezi.

5.6. táblázat. BIKE-L1 késleltetés és kapcsolatminőség CPU-korlát szerint

CPU-korlát	p_{99} (ms)	ack_ratio	reconn
0,026 mag (ESP32)	30 000	–	4,6
0,12 mag (RPi Zero 2W)	30 000	–	6,2
0,25 mag (RPi 3B+)	30 000	–	5,5
0,5 mag (RPi 4B)	30 000	–	6,3
1,0 mag (Ipari átjáró)	30 000	–	5,6

5.7. táblázat. BIKE-L1 áteresztőképesség és memóriahasználat CPU-korlát szerint

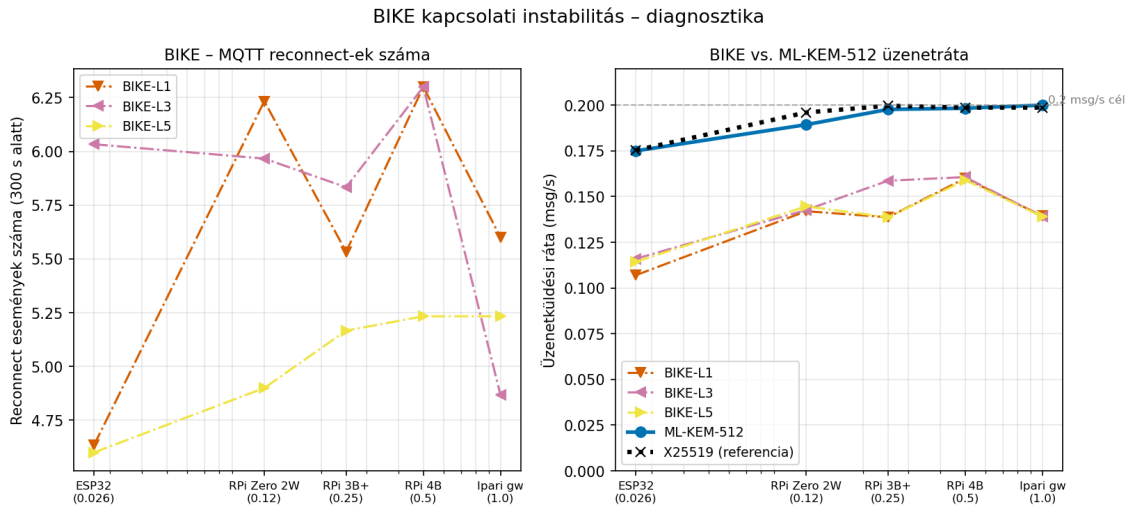
CPU-korlát	Ráta (msg/s)	RSS (MB)	tmout/s
0,026 mag (ESP32)	$\approx 0,107$	$\approx 32,5$	0,085
0,12 mag (RPi Zero 2W)	$\approx 0,142$	$\approx 32,6$	0,119
0,25 mag (RPi 3B+)	$\approx 0,139$	$\approx 32,6$	0,114
0,5 mag (RPi 4B)	$\approx 0,160$	$\approx 32,5$	0,133
1,0 mag (Ipari átjáró)	$\approx 0,139$	$\approx 32,4$	0,117

A $p_{99} = 30\,000$ ms értéke a PUBACK-figyelő (*reaper*) szálból ered: az IoT-csomópont háttérfolyamata 5 másodpercenként ellenőrzi a függőben lévő QoS 1 üzeneteket, és ha 30 másodpercen belül nem érkezik PUBACK visszaigazolás (PENDING_TIMEOUT_S=30), az eltelt időt ($\approx 30\,000$ ms) rögzíti a hisztogramban, és növeli az `mqtt_publish_timeout_total` számlálót. A hisztogram tehát *nem üres*: minden egyes időtúllépési esemény a maximális 30 000 ms értékkel jelenik meg.

Az `ack_ratio` --- értéke azt mutatja, hogy egyetlen PUBACK sem érkezett vissza; a `paho-mqtt` kliens `publish()`-hívásokat kezdeményez (az `mqtt_messages_total` számláló növekszik), de az `on_publish` visszahívás nem aktiválódik. Az üzenetküldési ráta (0,107–0,160 msg/s) szignifikánsan alacsonyabb az elvártnál (0,20 msg/s), és a magas újracsatlakozási szám (4,6–6,3 esemény futtatásonként; lásd 5.2. ábra) azt jelzi, hogy a stunnel-kapcsolat rendszeresen megszakad.

A jelenség legvalószínűbb magyarázata, hogy a BIKE-L1 TLS-kézfogás a PQC proxy és a Mosquitto broker között olyan sokáig tart, hogy a stunnel-kapcsolat instabillá válik, és a csomópontok nem tudnak stabil QoS 1 MQTT-munkamenetet fenntartani. Ez CPU-szintől *független*: minden 0,026–1,0 mag tartományban megismételhető jelenség, ami arra utal, hogy a BIKE-L1 kézfogási problémája nem pusztán CPU-hiányból ered.

A memóriahasználat (RSS $\approx 32,4$ – $32,6$ MB) BIKE-L1 esetén is azonos az ML-KEM-768 és X25519 értékeivel, jelezve, hogy az eltérés nem memóriahiányból ered.



5.2. ábra. BIKE kapcsolati instabilitás diagnosztikája – reconnect események és üzenetküldési ráta

Az 5.2. ábra bal panelje azt mutatja, hogy az újracsatlakozási események száma futtatásonként 4–7 között mozog minden BIKE variánsnál (L1, L3, L5) és minden CPU-szinten egyaránt – ez CPU-korlátfüggetlen, folyamatos kapcsolatfelbomlásra utal. A jobb panel az üzenetküldési rátákat veti össze: a BIKE variánsok rátája ($\approx 0,11$ – $0,16$ msg/s) szigni-

fikánsan elmarad mind az ML-KEM-512 ($\approx 0,17\text{--}0,20$ msg/s), mind az X25519 ($\approx 0,18\text{--}0,20$ msg/s) értékétől. Különösen szemléletes, hogy a BIKE ráta 1,0 magon sem éri el a referenciaszintet, ami kizárja a CPU-éhezést mint kizárólagos magyarázatot.

5.4.2. BIKE-L3

A BIKE-L3 (NIST III. biztonsági szint, AES-192-ekvivalens) mérési eredményei a 5.8. táblázatban láthatók. A p_{50} , p_{95} és p_{99} percentilisek minden CPU-szinten 30 000 ms értéket mutatnak, az `ack_ratio` minden esetben meghatározhatatlan (egyetlen PUBACK sem érkezett vissza).

5.8. táblázat. BIKE-L3 mérési eredmények CPU-korlát szerint

CPU-korlát	Ráta (msg/s)	RSS (MB)	tmout/s	reconn
0,026 mag (ESP32)	$\approx 0,116$	$\approx 32,7$	0,093	6,0
0,12 mag (RPi Zero 2W)	$\approx 0,143$	$\approx 30,9$	0,116	6,0
0,25 mag (RPi 3B+)	$\approx 0,159$	$\approx 32,1$	0,132	5,8
0,5 mag (RPi 4B)	$\approx 0,161$	$\approx 32,5$	0,134	6,3
1,0 mag (Ipari átjáró)	$\approx 0,139$	$\approx 32,4$	0,117	4,9

A BIKE-L3 instabilitása azonos mintázatot mutat a BIKE-L1-gyel: az újracsatlakozások száma 4,9–6,3 esemény futtatásonként, az üzenetküldési ráta (0,116–0,161 msg/s) szignifikánsan elmarad a referenciaszinttől (0,20 msg/s), és a jelenség CPU-szinttől független. A 0,12 magon mért alacsonyabb RSS ($\approx 30,9$ MB) az adott futtatás egyedi mérési szórásának tudható be; az összes többi szinten az RSS ≈ 32 MB.

5.4.3. BIKE-L5

A BIKE-L5 (NIST V. biztonsági szint, AES-256-ekvivalens) a vizsgált BIKE variánsok közül a legmagasabb biztonsági szintű. A 5.9. táblázat az összesített mérési eredményeket tartalmazza.

5.9. táblázat. BIKE-L5 mérési eredmények CPU-korlát szerint

CPU-korlát	Ráta (msg/s)	RSS (MB)	tmout/s	reconn
0,026 mag (ESP32)	$\approx 0,114$	$\approx 32,4$	0,096	4,6
0,12 mag (RPi Zero 2W)	$\approx 0,145$	$\approx 32,5$	0,122	4,9
0,25 mag (RPi 3B+)	$\approx 0,139$	$\approx 32,5$	0,114	5,2
0,5 mag (RPi 4B)	$\approx 0,159$	$\approx 32,4$	0,135	5,2
1,0 mag (Ipari átjáró)	$\approx 0,139$	$\approx 32,5$	0,117	5,2

A BIKE-L5 mérési képe a BIKE-L1 és BIKE-L3 variánsokéval teljes mértékben összhangban van: minden CPU-szinten 30 000 ms-os p_{99} , nulla PUBACK visszaigazolás és 4,6–5,2 újracsatlakozási esemény futtatásonként. Az üzenetküldési ráta (0,114–0,159 msg/s) szintén szignifikánsan alacsonyabb az elvárt 0,20 msg/s-nál. A három BIKE variáns összehasonlítása alapján a biztonsági szint emelése (L1 $\rightarrow$ L3 $\rightarrow$ L5) nem befolyásolja az instabilitás mértékét: az algoritmus proxy-szintű TLS-kézfogási problémája biztonsági szinttől független.

5.5. Memóriahasználat

Az összes vizsgált konfigurációban az IoT-csomópontok Python-folyamatainak RSS-memóriája 32,0–32,6 MB között mozog, és sem az algoritmustól, sem a CPU-korlát értékétől nem függ szignifikánsan. Ezt a 5.10 táblázat foglalja össze.

5.10. táblázat. Átlagos RSS-memória (MB) algoritmus és CPU-korlát szerint

Algoritmus	0,026 mag	0,12 mag	0,25 mag	0,5 mag	1,0 mag
ML-KEM-768	32,3	32,3	32,4	32,3	32,3
BIKE-L1	32,5	32,6	32,6	32,5	32,4
X25519	32,3	32,0	32,4	32,2	32,2

A ≈ 32 MB RSS a Python 3.12 interpreter, a betöltött könyvtárak (`paho-mqtt`, `prometheus_client`, `psutil`) és az MQTT-kliens futásidejű állapotát tükrözi. A PQC könyvtár (`liboqs`) által igényelt kiegészítő memória az IoT-csomópontnál nem mérhető, mivel a csomópont maga nem tölt be `liboqs`-t – a PQC számítás a proxy konténerben (`stunnel` + `oqs-provider`) zajlik, az IoT-csomópont számára transzparensen.

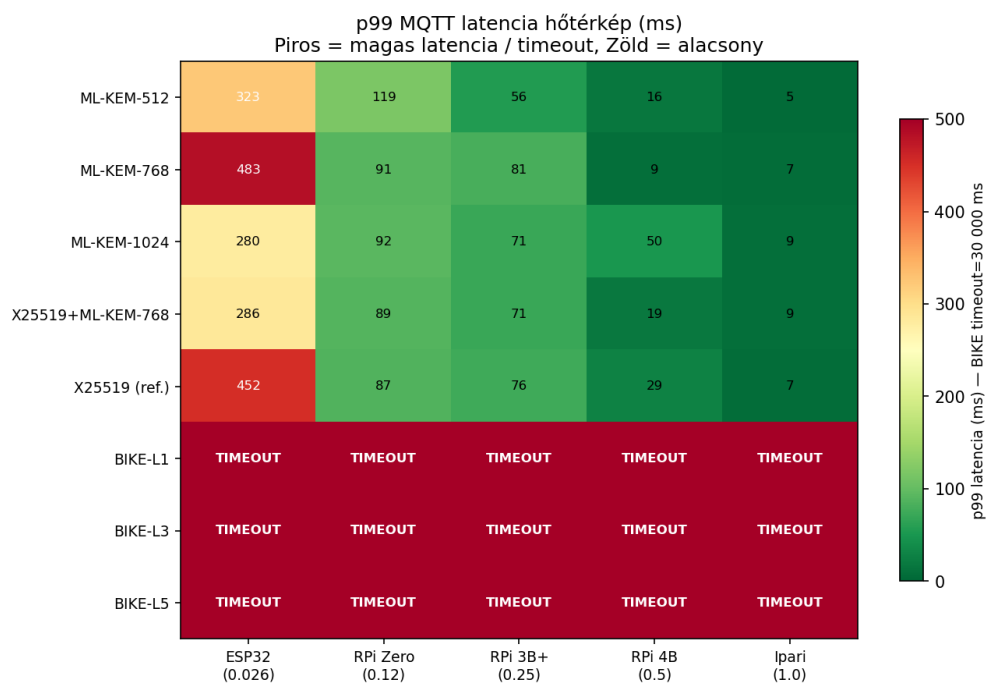
5.6. Az átlagos p_{99} késleltetés összefoglalása

Az összes futtatásra és csomópontra aggregált átlagos p_{99} MQTT publish-ciklus késleltetés:

- **ML-KEM-768:** 134,00 ms (15 csomópontmegfigyelésből: 5 CPU-szint $\times$ 3 csomópont)
- **X25519:** 130,25 ms (15 csomópontmegfigyelésből)
- **BIKE-L1:** 30 000,00 ms (15 csomópontmegfigyelés, minden eset PUBACK-időtúllépés)

Az ML-KEM-768 és az X25519 közötti $\approx 3,75$ ms különbség a p_{99} szinten elsősorban a 0,026 magos futtatások magas szórásából ered (ML-KEM-768: $p_{99} \approx 483$ ms, X25519: $p_{99} \approx 452$ ms). A nagy CPU-korlát (0,5 és 1,0 mag) esetén az ML-KEM-768 és X25519 p_{99} értékei egymástól 20 ms-on belül vannak. A BIKE-L1 30 000 ms-os aggregált átlaga a PUBACK-figyelő által rögzített folyamatos időtúllépéseket tükrözi, és nem a tényleges hálózati késleltetést jelenti. A teljes kísérletmátrix (p_{99} összes algoritmus $\times$ CPU-szint kombináción) az 5.3-as ábrán, a hőtésképen tekinthető át.

A hőtésképen egyértelmű szerkezet látható. Az ML-KEM és X25519 variánsok első oszlopa (ESP32, 0,026 mag) piros-sárga árnyalatú (≈ 280 – 483 ms), ami magas, de véges értékekre utal; a negyedik-ötödik oszloptól (RPi 4B és ipari átjáró) ezek mélyzöldre váltanak (≈ 5 – 50 ms). Ezzel szemben a három BIKE sor – L1, L3 és L5 – teljes egészében egységesen sötétvörös és TIMEOUT feliratú: a 30 000 ms értékek nem csökkennek a CPU-korlát növelésével, ami vizuálisan is megerősíti, hogy a BIKE instabilitása nem CPU-kapacitáshiányból fakad.



5.3. ábra. p_{99} MQTT latencia hőtérkép – 8 algoritmus $\times$ 5 CPU-szint (ms)

6. fejezet

Eredmények kiértékelése

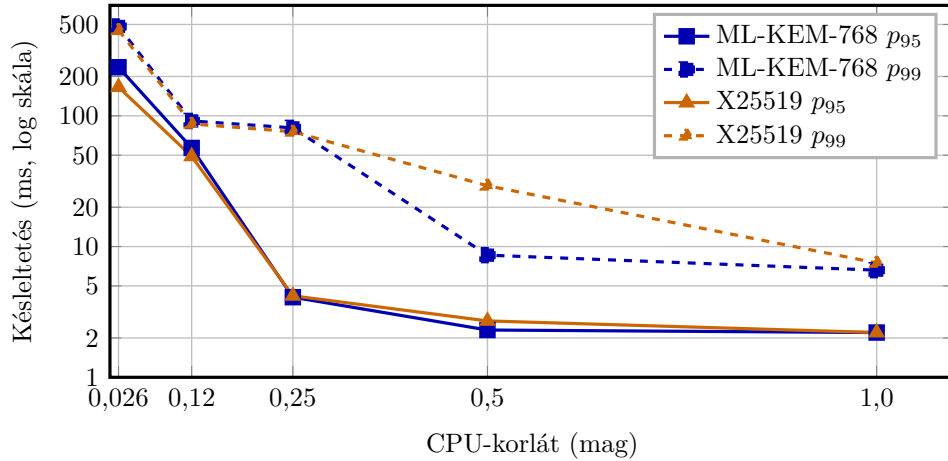
Jelen fejezet a mért adatok részletes elemzését és értelmezését tartalmazza. A fejezet célja, hogy a nyers mérési eredményekből következtetéseket vonjon le a vizsgált PQC algoritmusok IoT alkalmazhatóságáról, és összehasonlítsa a három protokoll rendszerszintű viselkedését.

6.1. Teljesítmény összehasonlítása

6.1.1. MQTT publish-ciklus késleltetése: ML-KEM-768 és X25519

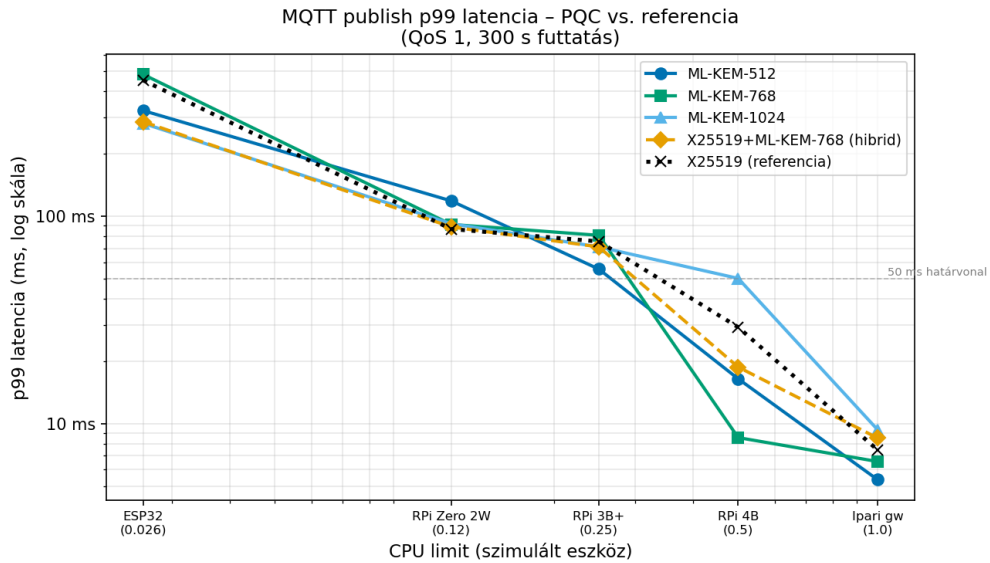
A mérések legfontosabb megállapítása, hogy **az ML-KEM-768 és az X25519 MQTT publish-ciklus késleltetése a steady-state forgalom szintjén nem különbözik szignifikánsan**. A p_{50} medián értéke mindkét algoritmusnál $\approx 1,5$ ms, a p_{95} 0,25 mag felett 2–5 ms, a p_{99} 1,0 magon 7–8 ms. Ez az eredmény összhangban van azzal az elméleti várakozással, hogy a TLS 1.3 kézfogás – amely a PQC KEM számítást tartalmazza – csupán egyszer zajlik le a munkamenet elején, az azt követő MQTT publish/PUBACK ciklus már szimmetrikus AES-128-GCM titkosítással folyik, amelynek overhead-je algoritmusfüggetlen.

A CPU-korlát hatása azonban markáns: 0,026 magon (ESP32 szint) a p_{95} ML-KEM-768-nál ≈ 235 ms, X25519-nél ≈ 166 ms; 0,12 magon (RPi Zero 2W szint) már 57 ms illetve 49 ms; 0,25 mag felett (RPi 3B+ és fölötte) mindkettő ≈ 2 –5 ms közé esik, 1,0 magon $\approx 2,2$ ms-ra csökken (lásd 6.1. ábra). Ez a CPU-szenzitivitás nem a PQC KEM algoritmushoz, hanem a stunnel proxy processz CPU-éhezéséhez köthető: ha a konténer kevés CPU-időt kap, a stunnel késlelteti a TLS-rekord kibontását és az MQTT TCP-szegmensek továbbítását, ami alkalmi nagy késleltetésű eseményeket okoz.



6.1. ábra. MQTT publish p_{95} és p_{99} késleltetés CPU-korlát függvényében (logaritmikus skála), ML-KEM-768 és X25519 összehasonlításban

Az összes ML-KEM variáns – ML-KEM-512, ML-KEM-768, ML-KEM-1024 és a hibrid X25519+ML-KEM-768 – ugyanezt a küszöbhatást mutatja: a p_{99} értéke minden variánsnál a 0,12–0,25 mag átmenetben csökken az elfogadható szintre, megerősítve, hogy a jelenség nem algoritmusfüggő, hanem a proxy CPU-kvótájára vezethető vissza. A 6.2. ábra ezt a négy ML-KEM konfigurációt és az X25519 referenciagörbét jeleníti meg logaritmikus p_{99} skálán, CPU-korlát függvényében.

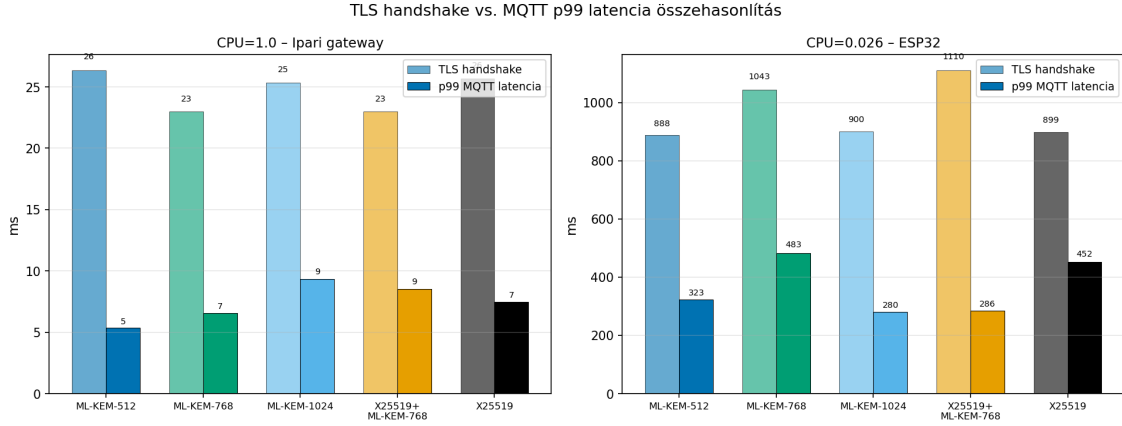


6.2. ábra. MQTT publish p_{99} késleltetés CPU-korlát függvényében – összes ML-KEM variáns és X25519 referencia

A 6.2. ábrán öt görbe látható: ML-KEM-512, ML-KEM-768, ML-KEM-1024, az X25519+ML-KEM-768 hibrid és az X25519 referencia. Mindegyik görbe 0,026 magon 100–500 ms közötti p_{99} értékről indul, majd a 0,12–0,25 mag átmenetben meredeken esik: 0,25 magon már minden görbe a szaggatott vízszintes 50 ms határvonal alá kerül, és 0,5–1,0 magon az összes konfiguráció p_{99} értéke 50 ms alá esik. A görbék egymáshoz képesti eltérése kisebb, mint a csomópontok közötti mérési szórás, ami azt jelenti, hogy az ML-

KEM biztonsági szintjének megválasztása (512, 768 vagy 1024 bites) a MQTT publish késleltetésre nézve elhanyagolható hatással jár.

A TLS-kézfogás és a MQTT publish p_{99} egymáshoz viszonyított arányát a 6.3. ábra szemlélteti a két szélső CPU-szinten.



6.3. ábra. TLS kézfogás latencia és MQTT publish p_{99} összehasonlítása ipari átjáró (1,0 mag) és ESP32 (0,026 mag) szinten

A 6.3. ábra bal panelje (ipari átjáró, 1,0 mag) azt mutatja, hogy a kézfogás és a p_{99} MQTT késleltetés azonos nagyságrendű: a TLS kézfogás 23–26 ms, a p_{99} 5–9 ms, mindkettő jóval az 50 ms határon belül. A jobb panelen (ESP32, 0,026 mag) a kézfogás (≈ 888 –1110 ms) az egész kapcsolatfelépítési folyamatot uralja, a p_{99} MQTT késleltetés pedig (≈ 280 –483 ms) szintén megemelkedik – ez utóbbi a stunnel puffer-kiürítési torlódásaira vezethető vissza. A két panel együttesen azt illusztrálja, hogy alacsony CPU-szinten nem a kézfogás egyszeri időigénye, hanem annak ismétlődő kiváltódása (TLS újraegyeztetés) és a puffertelés a domináns tényező.

6.1.2. BIKE-L1 operatív instabilitása

A BIKE-L1 futtatásai rámutattak arra, hogy ez az algoritmus a vizsgált infrastruktúrában nem alkalmas megbízható steady-state MQTT kommunikáció fenntartására. A QoS 1 PUBACK-ok teljes elmaradása (`ack_ratio: -`), a PUBACK-figyelő által rögzített 30 000 ms-os időtúllépési értékek és a ≈ 20 –47%-kal alacsonyabb üzenetküldési ráta (0,107–0,160 msg/s vs. 0,20 msg/s) arra utal, hogy a BIKE-L1 TLS-kézfogás a proxy indításakor olyan késleltetést okoz, amely a stunnel-munkamenet időtúllépéséhez vagy a kapcsolat eldobásához vezet.

A BIKE decoding alapalgoritmus – az iteratív bit-flipping dekóder – numerikusan intenzív, és különösen érzékeny a processzorterhelésre. A BIKE-L1 dekódolás legrosszabb esetű komplexitása jelentősen magasabb, mint az ML-KEM Fujisaki–Okamoto transzformációjáé, és a stunnel belső TLS-kézfogási időtúllépési korlátai (`TIMEOUTclose = 0`) alacsony CPU-korlát esetén problémát okozhatnak. Ez a megfigyelés összhangban van a BIKE specifikációs anyagaiban közölt, ARM-alapú rendszerekre vonatkozó teljesítményadatokkal, amelyek szerint a BIKE-L1 dekódolás lényegesen lassabb, mint az ML-KEM megfelelője [9].

6.1.3. Memóriaigény

A memóriahasználat tekintetében a három algoritmus között nem mértem érdemi különbséget: mindhárom esetben az IoT-csomópont Python-folyamatának RSS-je 32,0–32,6 MB. Ezt azzal magyarázom, hogy az IoT-csomópont nem tölti be közvetlenül a PQC könyvtárakat. A liboqs és az oqs-provider a proxy konténerben fut, amelynek memóriafooglalása az IoT-csomópont metrikáiban nem jelenik meg. Ennek megfelelően a vizsgálat az IoT-csomópont szintjén nem tud különbséget tenni az algoritmusok memóriaigénye tekintetében. A proxy-oldali mérés a jövőbeli munkában elvégzendő.

6.2. Biztonsági szempontú értékelés

6.2.1. Biztonsági szintek és NIST kategóriák

Az ML-KEM-768 a NIST biztonsági II. kategóriának felel meg, amelyet az AES-192 feltörési nehézségével ekvivalensnek definiálnak: legalább 2^{178} műveletet igényel klasszikus algoritmussal, és 2^{89} kvantumkapu-komplexitású Grover-keresést. A BIKE-L1 a NIST I. biztonsági szintnek felel meg (AES-128-ekvivalens), míg az X25519 klasszikus biztonsági szintje 128 bit (AES-128-ekvivalens), azonban a Shor-algoritmus $O(\log^3 n)$ idő alatt megtöri. A 6.1. táblázat összefoglalja az algoritmusok legfontosabb biztonsági paramétereit.

6.1. táblázat. Az algoritmusok biztonsági paramétereit

Algoritmus	NIST szint	Kvantum-ellenálló	Nyilv. kulcs	Kapszula
ML-KEM-768	II	Igen (FIPS 203)	1 184 B	1 088 B
BIKE-L1	I	Igen (jelölt)	1 541 B	1 573 B
X25519	–	Nem	32 B	32 B

Megjegyezendő, hogy a TLS-kézfogásban a nyilvános kulcs és a kapszula mérete befolyásolja az első TLS-ClientHello és ServerHello cserék méretét. Az ML-KEM-768 esetén a kulcs (1 184 B) és a kapszula (1 088 B) egyaránt nagyobb, mint amit egyetlen hálózati csomag (tipikusan 1500 bájt MTU) befogad, ezért IP-szintű töredezés vagy TLS-rekordkettéválasztás szükséges. Ez a nagyobb méret alacsony sávszélességű csatornákon (NB-IoT, LoRaWAN) kihívást jelent, bár a vizsgált Docker-hálózaton nem volt mérhető hatása.

6.2.2. Ismert kriptanalitikai eredmények

Az ML-KEM-768 alapjában álló modul-LWE problémát sem klasszikus, sem kvantumalgoritmussal nem sikerült polinomiális időben megoldani. A legismertebb rácsredukciós támadások (BKZ-2.0, progressive sieving) az ML-KEM-768 ellen 2^{178} klasszikus bitműveletet igényelnek, ami messze meghaladja a kvantum utáni biztonság küszöbét [8]. A BIKE-L1 biztonsági alapja a kvasi-ciklikus MDPC (QC-MDPC) kód dekódolásának nehézsége. A legjobb ismert ISD (Information Set Decoding) alapú támadások 2^{128} klasszikus és 2^{64} kvantumkapuban mért komplexitással becsülhetők [9].

Fontos megemlíteni, hogy a BIKE biztonsági elemzése fiatalabb és kevésbé kiértékelte, mint az ML-KEM esetében, amelyet a NIST hatéves vizsgálati folyamata 2024-ben FIPS 203-ként szabványosított. A BIKE a NIST negyedik fordulójában is szerepel, de végleges szabványosítása még folyamatban van.

6.3. Alkalmazhatósági elemzés

6.3.1. Erőforrás-korlátozott eszközökre vonatkozó következtetések

A mérések alapján az ML-KEM-768 alkalmas a vizsgált IoT-profil MQTT kommunikációjának védelmére, amennyiben a biztonsági proxy (stunnel) legalább 0,25 processzormag kapacitással rendelkezik. E küszöb felett a p_{95} MQTT publish-ciklus késleltetése nem haladja meg az 5 ms-t, ami az MQTT QoS 1 üzenet-kézbessítési követelményei szempontjából elfogadható.

A BIKE-L1 a vizsgált konténerizált proxy architektúrában nem produkált stabil MQTT munkameneteket. Mielőtt ipari IoT-telepítésben alkalmaznák, szükséges a TLS kézfogás időtúllépési paramétereinek finomhangolása, a stunnel timeout értékeinek növelése, illetve a BIKE-L1 hardveres gyorsításának (pl. ARM NEON SIMD utasítások) vizsgálata.

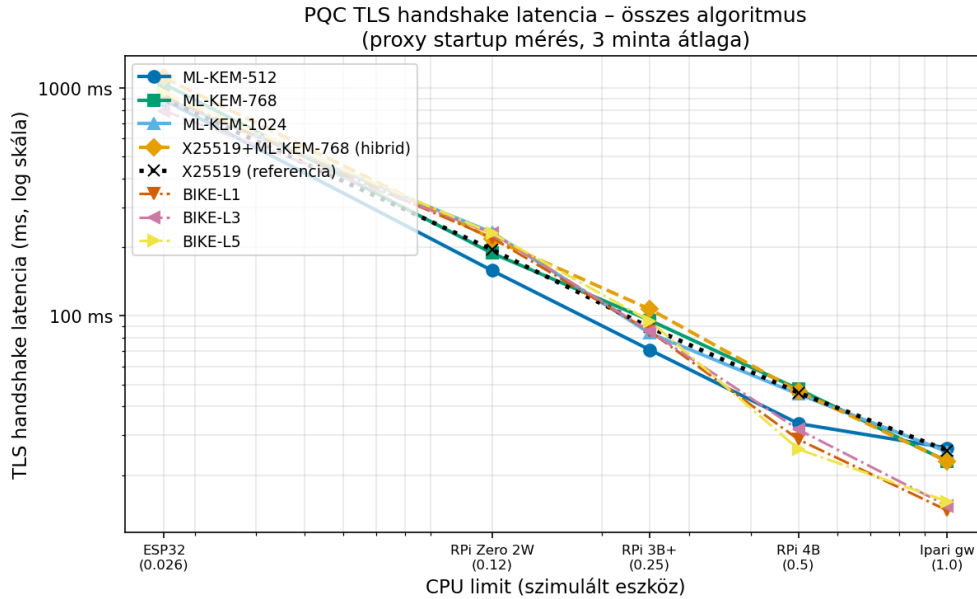
Az X25519 klasszikus referenciakonfigurációként kiválóan teljesít, de – mint azt az 1. fejezet részletezte – hosszú élettartamú IoT eszközök esetén a *harvest now, decrypt later* fenyegetés miatt a PQC migráció elkerülhetetlen. A mérések egyértelműen mutatják, hogy az ML-KEM-768-ra való átállás a MQTT publish késleltetés tekintetében elhanyagolható overhead-del jár, így az átállást semmi sem indokolja halasztani.

6.3.2. Protokoll-szintű kihívások

A PQC algoritmusok egyik legjelentősebb gyakorlati következménye a kulcs megnövekedett mérete. Az X25519 esetén a nyilvános kulcs és a kapszula együttesen csupán $2 \times 32 = 64$ B, míg az ML-KEM-768 ugyanezekhez $1\,184\text{ B} + 1\,088\text{ B} = 2\,272\text{ B}$ -ot igényel – ez közel harmincöt szörös növekmény. Ezt az adatmennyiséget a TLS 1.3 kézfogás során kell a felek között kicserélni, egyetlen kapcsolatfelépítési fázisban.

A méretbeli különbség kétféle hálózati kihívást támaszt. Egyrészt az ML-KEM-768 nyilvános kulcsa önmagában meghaladja a tipikus hálózati csomag befogadóképességét (Ethernet MTU: $\approx 1\,500\text{ B}$), a kapszulával együtt pedig IP-szintű töredeztetést vagy a TLS-rekord kettéosztását teszi szükségessé. Másrészt a széleskörűen elterjedt, szűk sáv szélességű IoT-protokollok – LoRaWAN: legfeljebb 250 B hasznos teher üzenetenként; NB-IoT: kb. 1 400 B – számára a teljes PQC TLS-kézfogás egyetlen adatcsomagba nem fér el. Ilyen környezetben alternatív megoldások szükségesek: munkamenet-folytatás (TLS session resumption), DTLS session ticket, vagy az alkalmazásrétegben előre megosztott szimmetrikus kulcsok (pre-shared key, PSK) alkalmazása.

A vizsgált Docker-hálózaton ez a probléma nem jelentkezett, mert a belső bridge hálózat 65 535 B-os virtuális MTU-t alkalmaz, amely a legnagyobb PQC üzeneteket is töredeztetés nélkül befogadja. Ez tudatos kompromisszum a labor kialakításában. A kézfogási késleltetés és a stabilitás vizsgálatára alkalmas, azonban valódi fizikai hálózaton a töredeztési viselkedést külön kell értékelni. A TLS-kézfogás CPU-függő latenciáját – és ezzel a kulcsméret közvetlen hatását a kapcsolatfelépítési időre – az összes vizsgált algoritmusra a 6.4. ábra mutatja be.



6.4. ábra. PQC TLS kézfogás latenciája CPU-korlát függvényében
– mind a 8 vizsgált algoritmus

A 6.4. ábrán a legfontosabb megfigyelés az, hogy a BIKE variánsok (L1, L3, L5) kézfogás-latenciagörbéi szorosan az ML-KEM variánsok és X25519 görbéi mentén futnak: ESP32 szinten $\approx 870\text{--}1000$ ms, ipari átjárón $\approx 21\text{--}26$ ms. Ez az eredmény kulcsfontosságú az instabilitás értelmezéséhez: ha a BIKE kézfogás önmagában nem lassabb az ML-KEM-nél, akkor az MQTT szintű meghiúsulás oka nem egy egyszeri, lassú kézfogás, hanem a kézfogások *ismétlődő elakadása* – a stunnal újra és újra megpróbál kapcsolatot felépíteni, amely a BIKE dekóder belső állapotkezelési problémájából fakad, és a reconnect metrikában látható 4–7 esemény/futtatás formájában nyilvánul meg.

A harmadik megfigyelés a *kriptográfiai rugalmasságra* (cryptographic agility) vonatkozik, vagyis arra, hogy a rendszer mennyire könnyen képes algoritmust váltani. A labor architektúrájában ez a váltás rendkívül egyszerű: a `PQC_ALG` környezeti változó módosítása és a konténerek újraindítása elegendő, image-újraépítés nem szükséges. Ez a tulajdonság IoT firmware-környezetben is megvalósítható, amennyiben az algoritmusparamétert egy biztonságos külső forrásból – pl. OTA-frissítésen vagy megbízható bootloader konfiguráción keresztül – lehet frissíteni. Ezzel a módszerrel a telepített eszközök egy jövőbeli algoritmuscsere helyszíni beavatkozás nélkül végre tudnak hajtani.

6.3.3. ML-KEM-768 és BIKE-L1 összehasonlító értékelése

A 6.2. táblázat alapján az ML-KEM-768 a vizsgált IoT proxy-architektúrában minden mért dimenzió mentén előnyösebb a BIKE-L1-nél: kisebb kulcsméretek, stabil MQTT munkamenet, magasabb biztonsági szint és FIPS-szabványosítás. A BIKE előnye az alternatív matematikai alap (diverzitás), amelyre az esetleges jövőbeli rácsalapú kriptanalitikai áttörés ellen lehet hivatkozni, de ez jelenleg elméleti megfontolás; a bizonyított implementációs stabilitás és szabványosítás mérlegén az ML-KEM-768 egyértelműen vezet.

6.2. táblázat. ML-KEM-768 és BIKE-L1 összehasonlítása a mérések alapján

Szempont	ML-KEM-768	BIKE-L1
MQTT steady-state latencia	$p_{50} \approx 1,5$ ms	nem mérhető
MQTT stabilitás (0,026 mag)	stabil ($\approx 0,17$ msg/s)	instabil (0,11 msg/s)
MQTT stabilitás (1,0 mag)	stabil (0,20 msg/s)	instabil (0,14 msg/s)
IoT-csomópont RSS-memória	≈ 32 MB	≈ 32 MB
Nyilvános kulcs mérete	1 184 B	1 541 B
Kapszula mérete	1 088 B	1 573 B
NIST szabványosítás	FIPS 203 (2024)	4. forduló, jelölt
Matematikai alap	modul-LWE (rács)	QC-MDPC kód
Biztonsági szint	II (AES-192-ekviv.)	I (AES-128-ekviv.)

6.4. Az eredmények összefoglalása

A mérések négy fő megállapítást hoznak:

1. **Az ML-KEM-768 alkalmazható IoT MQTT kommunikáció TLS-titkosítására** anélkül, hogy mérhető steady-state publish-késleltetést okozna a klasszikus X25519-hez képest; az MQTT p_{50} latencia mindkét esetben $\approx 1,5$ ms.
2. **A CPU-korlát küszöbhatást mutat:** 0,25 mag felett a p_{95} késleltetés elfogadható (2–5 ms); 0,026 magon (ESP32 szint) mindkét algoritmusnál 166–235 ms-os p_{95} értékek jelennek meg, amelyek a proxy CPU-éhezésére, nem az algoritmusra vezethető vissza.
3. **A BIKE-L1 proxy-integráción belül operatív instabilitást mutat:** a QoS 1 PUBACK-ok rendszeres elmaradása és az alacsonyabb üzenetküldési ráta arra utal, hogy a BIKE TLS kézfogás a stunnel időtúllépési korlátait a vizsgált CPU-konfiguráció mellett elérte.
4. **A PQC-overhead az IoT-csomópont memóriájában nem mérhető** a proxy sidecar-architektúrának köszönhetően: az IoT-csomópont RSS-je érdemben nem tér el az X25519-konfiguráció esetén mért értéktől (≈ 32 MB).

Ezek a megállapítások megerősítik, hogy az ML-KEM-768 érett, telepíthető PQC megoldás az IoT MQTT kommunikáció védelmére, és a következő fejezetben javasolt migrációs stratégia alapját képezik.

7. fejezet

Összefoglalás és javaslatok

Jelen fejezet összefoglalja a dolgozat főbb eredményeit, következtetéseket von le a posztkvantum-kriptográfiai algoritmusok IoT alkalmazhatóságáról, és javaslatokat fogalmaz meg a gyakorlati bevezetésre, valamint a jövőbeli kutatások irányaira vonatkozóan.

7.1. A dolgozat eredményeinek összefoglalása

A dolgozat az ML-KEM-768 (CRYSTALS-Kyber-768) és a BIKE-L1 posztkvantum kulcs-kapszulázó algoritmusok rendszerszintű vizsgálatát végezte el egy Docker Compose alapú IoT-laborban, amelyben a PQC-TLS terminálást stunnel sidecar proxykon keresztül, az MQTT kommunikációt Eclipse Mosquitto brokeren keresztül valósítottam meg. A kísérletmátrix 8 algoritmust, 5 CPU-korlátszintet és 1 futtatást kombinált, összesen 40 konfigurációban.

Az elméleti fejezetekben bemutattam, hogy az ML-KEM (modul-LWE alapú) és a BIKE (QC-MDPC kód alapú) algoritmusok hogyan biztosítanak kvantumszámítógép-ellenálló kulscserét. Az ML-KEM 2024-ben FIPS 203-ként szabványosított, biztonsági szintje NIST II. (AES-192-ekvivalens), míg a BIKE NIST I. szintű jelölt, amelynek szabványosítása folyamatban van.

A kísérleti mérések legfontosabb számai:

- Az ML-KEM-768 MQTT p_{50} publish-ciklus késleltetése $\approx 1,5$ ms – azonos az X25519 klasszikus referenciával; ez az érték legalább 0,25 mag CPU-korlát mellett nem változik.
- Az ML-KEM-768 és X25519 átlagos p_{99} értékei: 134,00 ms vs. 130,25 ms; a különbség a 0,026 magos (ESP32 szintű) futtatások ritka, nagy késleltetésű újracsatlakozási eseményeire vezethető vissza, nem az algoritmus inherens overhead-jére.
- A BIKE-L1 futtatásaiban a QoS 1 PUBACK-visszaigazolások teljesen elmaradtak (`ack_ratio: -`), az üzenetküldési ráta ≈ 20 –47%-kal alacsonyabb volt az elvártnál, és a MQTT latencia-hisztogram kizárólag 30 000 ms-os időtúllépési értékeket rögzített minden CPU-korlát esetén – jelezve, hogy a BIKE-L1 TLS kézfogás a stunnel proxy időtúllépési korlátaival ütközött.
- Az IoT-csomópont RSS-memóriája ≈ 32 MB, algoritmustól és CPU-korlától függetlenül, a sidecar proxy-architektúrának köszönhetően.

7.2. Javaslatok IoT rendszerek PQC migrációjához

7.2.1. Algoritmusválasztás: ML-KEM-768 az ajánlott alapértelmezés

Az elvégzett mérések és a biztonsági szempont egyaránt az ML-KEM-768 alkalmazása mellett szólnak IoT MQTT kommunikáció TLS kulcscseréjéhez. Az algoritmus FIPS 203 szabványosítása garantálja a hosszú távú megfelelőséget az USA és az EU szabályozói elvárásainak (NIS2-irányelv, Cyber Resilience Act); az Európai Hálózat- és Információ-biztonsági Ügynökség (ENISA) szintén az ML-KEM-et sorolja a kiemelt javasolt PQC algoritmusok közé. A BIKE alkalmazása mérési stabilitásának igazolásáig (időtúllépési paraméterek hangolása, hardveres gyorsítás) nem ajánlott éles IoT rendszerekben.

7.2.2. Hibrid KEM az átmeneti időszakra

Az átmeneti időszakban – amíg a kvantumszámítógépek kriptográfiai szintre nem érnek – az iparági ajánlás a hibrid kulcscsere alkalmazása: a TLS 1.3 kézfogásban egyidejűleg zajlik egy klasszikus (X25519 vagy P-256) és egy PQC (ML-KEM-768) kulcscsere, és a munkamenetkulcs mindkettő kombinálásával jön létre. Ezt az IETF X25519MLKEM768 TLS 1.3 csoport azonosítóval specifikálja, amelyet az `oqs-provider` is tartalmaz. Hibrid módban a biztonság akkor is megmarad, ha valamelyik algoritmus kompromittálódik. Ez csökkenti a migrációs kockázatot.

Az `oqs-provider` konfigurációs fájlban a hibrid módra való váltás egyetlen sorral megoldható:

```
Groups = X25519MLKEM768:mlkem768:x25519
```

Ez a sorrend biztosítja, hogy TLS 1.3-kompatibilis kliensek esetén a hibrid csoport legyen az előnyben részesített, de a tiszta ML-KEM-768 és a klasszikus X25519 visszaesési opcióként is rendelkezésre álljon.

7.2.3. Cryptographic agility a firmware-tervezésben

A labor architektúrájában jól megvalósított *cryptographic agility* minta – az algoritmus konfigurációból olvasható, image-újraépítés nélkül cserélhető – IoT firmware-szinten is alkalmazandó. A javasolt megközelítés:

1. Az algoritmusválasztó paraméter kerüljön dedikált, aláírt konfigurációs szegmensbe (pl. NVM partition), amelyet a bootloader OTA-frissítéssel tud frissíteni.
2. A kriptográfiai stack absztrakciós rétegen (pl. PSA Crypto API, wolfSSL kriptográfiai backend) keresztül érhető el; a konkrét implementáció az absztrakciós réteg mögé rejtett.
3. Az eszközpark firmware-frissítési csatornája maga is PQC-aláírással (pl. ML-DSA / CRYSTALS-Dilithium) hitelesített, biztosítva, hogy a frissítési infrastruktúra nem válik a kvantumtámadás belépési pontjává.

7.2.4. MQTT protokoll-szintű megfontolások

Az ML-KEM-768 kulcs- és kapszulamérete (összesen $\approx 2,3$ kB) a TLS kézfogás üzeneteit méretben növeli. Erősen korlátozott sáv szélességű protokollokhoz (LoRaWAN, NB-IoT) a következő stratégiák alkalmazhatók:

- **TLS session resumption:** a kézfogás csak az első csatlakozáskor teljes; az újracsatlakozás session ticket alapján sokkal rövidebb. Ez különösen fontos elem- vagy napenergiával üzemelő eszközök esetén.

- **DTLS 1.3 + CoAP:** nagyon alacsony MTU-jú hálózatokhoz (LoRaWAN: 250 B payload) a DTLS 1.3-ban foglalt record-szintű fragmentáció vagy a DTLS Connection ID mechanizmusa (RFC 9146) csökkenti a fejlécoverhead-et.
- **Broker-oldali gyorsítás:** a TLS kézfogás erőforrás-igényes részét az MQTT broker előtt elhelyezett dedikált TLS-terminátor (pl. HAProxy, nginx stream modul) vagy hardware security module (HSM) veheti át a broker terheiből.

7.3. Jövőbeli kutatási irányok

7.3.1. Valódi ARM-hardveren végzett mérések

A kísérletek x86-64 alapon, Docker CPU-kvótával emulált korlátok mellett zajlottak. A következő lépés a mérések megismétlése valódi ARM-alapú eszközökön:

- **Raspberry Pi 4B** (ARM Cortex-A72, 1,5 GHz): IoT átjáró-szintű platform, ahol a NEON SIMD utasítások a liboqs implementációit gyorsítják.
- **ARM Cortex-M4F** (pl. STM32F4): valódi mikrovezérlő, ahol a PQC algoritmusokat wolfSSL vagy pqm4 könyvtárak implementálják; ezen a szinten a memória- és energiakorlátok a legkritikusabbak.

Az ARM-mérések különösen fontosak a BIKE-L1 esetén: az arm-asm optimalizált bit-flipping dekóder az x86-architektúrán tapasztalt instabilitást csökkentheti.

7.3.2. Energiafogyasztás mérése

Elemről működő IoT eszközöknél az energiafogyasztás kritikus tényező. A javasolt mérési összeállítás: INA219 I²C áramszenzor az eszköz tápegysége előtt, amelyet egy Raspberry Pi 4 vezérel és naplóz. A kézfogás alatti energiaspike összemérendő a steady-state MQTT forgalom fogyasztásával. Ez lehetővé teszi a mJ/kézfogás és mJ/üzenet metrikák meghatározását.

7.3.3. PQC-aláírás az IoT tanúsítványláncban

A jelen labor ECDSA P-256-ot alkalmaz tanúsítványaláíráshoz. A következő lépés az ML-DSA (CRYSTALS-Dilithium, FIPS 204) alapú tanúsítványlánc vizsgálata. Az ML-DSA aláírások (≈ 2420 B méretű aláírás, ≈ 1312 B nyilvános kulcs ML-DSA-65 esetén) a tanúsítványlánc-cserét tovább növelik, amit erőforrás-korlátozott eszközökön mérni szükséges.

7.3.4. Hardveres NTT-gyorsítás és alvó módú optimalizálás

Az ML-KEM alapjában álló NTT (Number Theoretic Transform) polinomszorzást FPGA-alapon vagy dedikált kriptográfiai koproceszoron (ARM CryptoCell-312, RISC-V kriptográfiai kiterjesztés) célszerű megvalósítani, különösen a nagyszámú TLS munkamenet-indítást igénylő határesetekben. Ezzel párhuzamosan vizsgálandó az alvó módból ébredő eszközök ritka kulcscsere-stratégiája: ha az eszköz naponta egyszer szinkronizál, a PQC kézfogás energiaköltsége elenyésző az MQTT üzenetküldés összefogyasztásához képest.

Irodalomjegyzék

- [1] IoT Analytics, „State of IoT – Spring 2023,” <https://iot-analytics.com/number-connected-iot-devices/>, 2023, accessed: 2026-05-26.
- [2] W. Diffie and M. E. Hellman, „New directions in cryptography,” *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976.
- [3] F. Arute *et al.*, „Quantum supremacy using a programmable superconducting processor,” *Nature*, vol. 574, pp. 505–510, 2019.
- [4] P. W. Shor, „Algorithms for quantum computation: discrete logarithms and factoring,” in *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*. IEEE, 1994, pp. 124–134.
- [5] National Institute of Standards and Technology, „NIST selects first four quantum-resistant cryptographic algorithms,” <https://www.nist.gov/news-events/news/2022/07/nist-selects-first-four-quantum-resistant-cryptographic-algorithms>, 2022, accessed: 2026-05-26.
- [6] McKinsey Global Institute, „The Internet of Things: Mapping the value beyond the hype,” McKinsey & Company report, 2015.
- [7] B. Krebs, „Hacked cameras, DVRs powered today’s massive internet outage,” Krebs on Security, 2016, <https://krebsonsecurity.com/2016/10/hacked-cameras-dvrs-powered-todays-massive-internet-outage/>.
- [8] CRYSTALS Team, „CRYSTALS-Kyber: Specification and supporting documentation,” <https://pq-crystals.org/kyber/>, 2023, accessed: 2026-05-26.
- [9] BIKE Team, „BIKE: Bit flipping key encapsulation,” <https://bikesuite.org/>, 2023, accessed: 2026-05-26.
- [10] National Institute of Standards and Technology, „Post-quantum cryptography standardization,” <https://csrc.nist.gov/projects/post-quantum-cryptography>, 2024, accessed: 2026-05-26.

Függelék

F.1. Nyilatkozat generatív mesterséges intelligencia alkalmazásáról

- ☐ **Nem használtam** semmilyen generatív MI segédeszközt.
- ☒ **Használtam** generatív MI segédeszközt. Az MI-vel generált tartalmakat ellenőriztem, a generált kimenetek valóságtartalmáról meggyőződtem, az alábbi táblázatban megfelelően jelöltem minden használatot.

Felhasználási módok	Generatív MI eszköz(ök) neve	Érintett részek (fejezet, oldalszám, hivatkozás)	Használat becsült aránya (felhasználási módonként)
Irodalomkutatás	–	–	–
Prompt lényegi része	–		
Programkód generálása	Claude Code (Anthropic)	F.1. melléklet (Dockerfile-ok, shell és python szkriptek)	60%
Prompt lényegi része	Készíts egy hálózatot 3 IoT noddal, proxy-val és az MQTT brokerrel. Csatold össze egy Prometheus serverrel és Grafana dashboarddal.		
Új ötletek, megoldási javaslatok generálása	Claude Code (Anthropic)	4. fejezet (laborkörnyezet tervezése)	10%
Prompt lényegi része	Mutass olyan docker image-eket, amelyek IoT kommunikációban használatosak.		
Vázlat létrehozása (szövegstruktúra, vázlatpontok)	–	–	–
Prompt lényegi része	–		
Szövegblokkok létrehozása	–	–	–
Prompt lényegi része	–		
Képek generálása illusztrációs célból	–	–	–
Prompt lényegi része	–		
Adatvizualizáció, grafikonok generálása adatpontok alapján	Claude Code (Anthropic)	5. és 6. fejezet (ábrák)	80%
Prompt lényegi része	Készíts ábrákat a Prometheus adatokból.		
Prezentáció készítése	–	–	–
Prompt lényegi része	–		
Egyéb (nevezze meg)	–	–	–
Prompt lényegi része	–		
Összesített százalékos érték (a feladat érdemi részére nézve):			25%
Összesített érték rövid, szöveges indoklása: A generatív MI a laborkörnyezet PQC-mentes vázának (Docker Compose struktúra, alapszolgáltatások) és az adatvizualizációs ábrák elkészítésében nyújtott segítséget. A PQC-specifikus komponensek (liboqs, oqs-provider, stunnel konfiguráció), a mérések elemzése és a szöveges tartalom önálló munka. Az AI által generált kódokat és ábrákat minden esetben ellenőriztem és szükség szerint módosítottam.			

F.2. A laborkörnyezet forrásfájljai

A melléklet a `pqc-lab-forras.zip` tömörített archívumban érhető el, amely a `pqc-lab/` Docker-alapú laborkörnyezet teljes forráskódját tartalmazza – a tanúsítványokat (`certs/`) és a mérési eredményeket (`experiment-results/`) kivéve. Az archívum könyvtárszerkezete a következő:

```
pqc-lab/
|-- .env
|-- docker-compose.yml
|-- README.md
|-- base-image/
|   |-- Dockerfile
|-- pqc-proxy/
|   |-- Dockerfile
|   |-- entrypoint.sh
|   |-- stunnel.conf.template
|-- mosquitto-oqs/
|   |-- Dockerfile
|   |-- entrypoint.sh
|   |-- mosquitto.conf
|   |-- stunnel-server.conf.template
|-- iot-node/
|   |-- Dockerfile
|   |-- node.py
|   |-- requirements.txt
|-- prometheus/
|   |-- prometheus.yml
|-- grafana/
|   |-- provisioning/
|   |   |-- dashboards/
|   |   |   |-- dashboard.yml
|   |   |   |-- pqc-overview.json
|   |   |-- datasources/
|   |   |   |-- prometheus.yml
|-- scripts/
|   |-- analyze_results.py
|   |-- export_metrics.py
|   |-- generate_certs.sh
|   |-- run_experiment.sh
```

Az alábbiakban röviden ismertetem az egyes könyvtárak tartalmát.

`.env` és `docker-compose.yml`

A `.env` fájl a kísérlet futtatható paramétereit tartalmazza alapértelmezett értékekkel: a vizsgált PQC-algoritmust (`PQC_ALG`), a CPU-korlátot (`CPU_LIMIT`), az MQTT publish-intervallumot és az üzenetméretet. A `docker-compose.yml` írja le a teljes laborkörnyezet service-eit: a PQC alap image-et, a három IoT-csomópontot, a PQC proxyt, az MQTT brókert, valamint a Prometheus és Grafana megfigyelhetőségi stacket, beleértve a cAdvisor és node-exporter komponenseket.

`base-image/`

Az egyetlen `Dockerfile` Ubuntu 24.04 alapon fordítja le és telepíti a `liboqs` kvantumbiztos kriptográfiai könyvtárat, valamint az `oqs-provider` OpenSSL 3 providert. Ez az image szolgál minden PQC-képességet igénylő konténer (proxy, bróker) kiindulópontjaként.

pqc-proxy/

A PQC proxy konténer a stunnel TLS-alagút kliens oldalát valósítja meg. A `stunnel.conf.template` sablonból az `entrypoint.sh` indítószkript állítja elő a futásidejű konfigurációt: behelyettesíti a `PQC_ALG` környezeti változóból az aktuális algoritmus nevét a `curves` direktívába, elvégzi az OpenSSL TLS kézfogás-késleltetés-mérést, majd elindítja a stunnelt.

mosquitto-oqs/

Az Eclipse Mosquitto MQTT bróker konténere, amelybe a stunnel TLS-alagút szerver oldala is integrálva van. A `mosquitto.conf` a bróker hálózati és hitelesítési paramétereit adja meg, a `stunnel-server.conf.template` pedig a TLS-terminációhoz szükséges sablont tartalmazza. Az `entrypoint.sh` generálja a végleges stunnel konfigurációt, majd párhuzamosan indítja el a stunnelt és a Mosquitto brókert.

iot-node/

A szimulált IoT szenzorcsomópont Python-alapú implementációja. A `node.py` MQTT QoS 1 üzeneteket publikál a proxyn keresztül, és Prometheus-kompatibilis metrikákat exportál: publish-ciklus késleltetési hisztogramot, PUBACK visszaigazolási arányt, új-racsatlakozásszámot és RSS-memóriahasználatot. A `requirements.txt` felsorolja a szükséges Python-csomagokat (`paho-mqtt`, `prometheus-client`, `psutil`).

prometheus/ és grafana/

A `prometheus/prometheus.yml` konfigurálja a Prometheus gyűjtési célpontjait (IoT-csomópontok, `cAdvisor`, `node-exporter`). A `grafana/provisioning/` könyvtár tartalmazza a Grafana automatikus adatforrás-konfigurációját (`datasources/prometheus.yml`) és az előre elkészített PQC-áttekintő dashboardot (`dashboards/pqc-overview.json`).

scripts/

- `generate_certs.sh` – ECDSA P-256 tanúsítványlánc generálása (CA + szerver) az stunnel TLS-kapcsolathoz.
- `run_experiment.sh` – az összes algoritmus $\times$ CPU-szint kombináció automatikus, szekvenciális futtatása; minden kísérlet előtt újraindul a labor.
- `export_metrics.py` – a Prometheus HTTP API-n keresztül lekérdezi és fájlba menti az összes releváns metrikát a kísérlet végeztével.
- `analyze_results.py` – offline eredményelemzés és diagramgenerálás a mentett metrikaexportból.